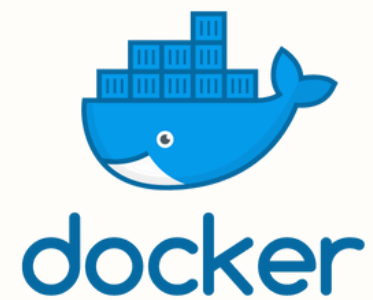




ATELIER

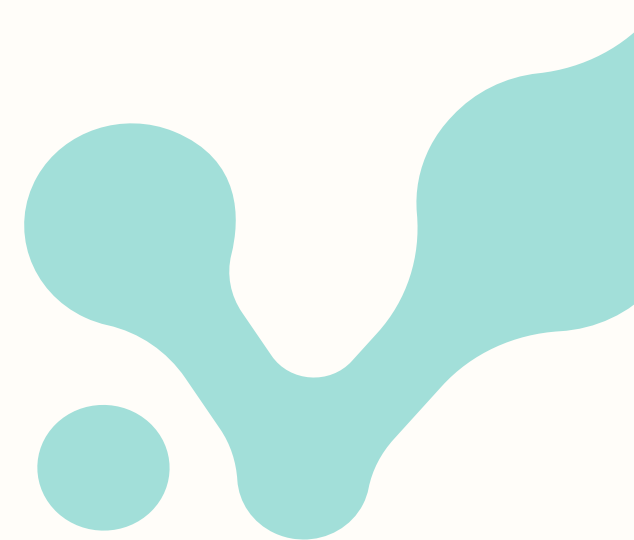
CONTENEURISATION

AVEC DOCKER & CI/CD

Présenté par : Azzman Nor Elimane
EL GHAZI Silia
Lio Eddine Samia



Sommaire de la Session – Docker & CI/CD DevOps



Partie 1 – Présentation Théorique

1. Introduction à Docker → Concepts clés, architecture, commandes, Dockerfile simple
2. Docker dans le workflow développeur → docker-compose, .env, volumes, debug, bonnes pratiques
3. Docker & CI/CD dans un projet réel → GitHub Actions, DockerHub, optimisations, déploiement
4. Conclusion commune & Q/R

Partie 2 – Workshop Pratique

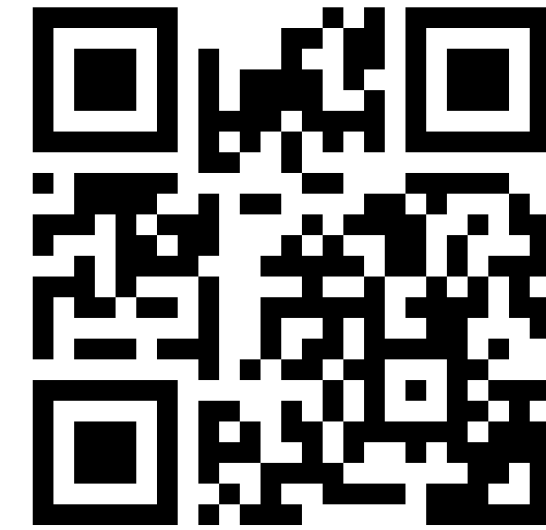
1. Préparation du proje → Clonage, structure, configuration .env
2. Dockerisation des services → Backend Spring Boot, Frontend React, PostgreSQL, docker-compose
3. CI/CD avec GitHub Actions → Création du pipeline, push Docker image, déploiement
4. Test & Validation → Accès app, vérification API + DB, logs, debug



Matériel & Ressources



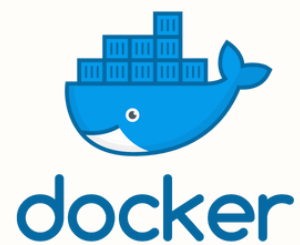
- Repo Git (base du projet)



- Accès DockerHub

- avoir un repo personnel pour CI.CD
- Fichier .env.example
- Quiz/QCM (optionnel)





PARTIE 1

INTRODUCTION À DOCKER



Docker à la Rescue

Docker est une solution.

Selon vous, quels problèmes Docker résout-il ?



**Incohérence des
environnements**

Portabilité

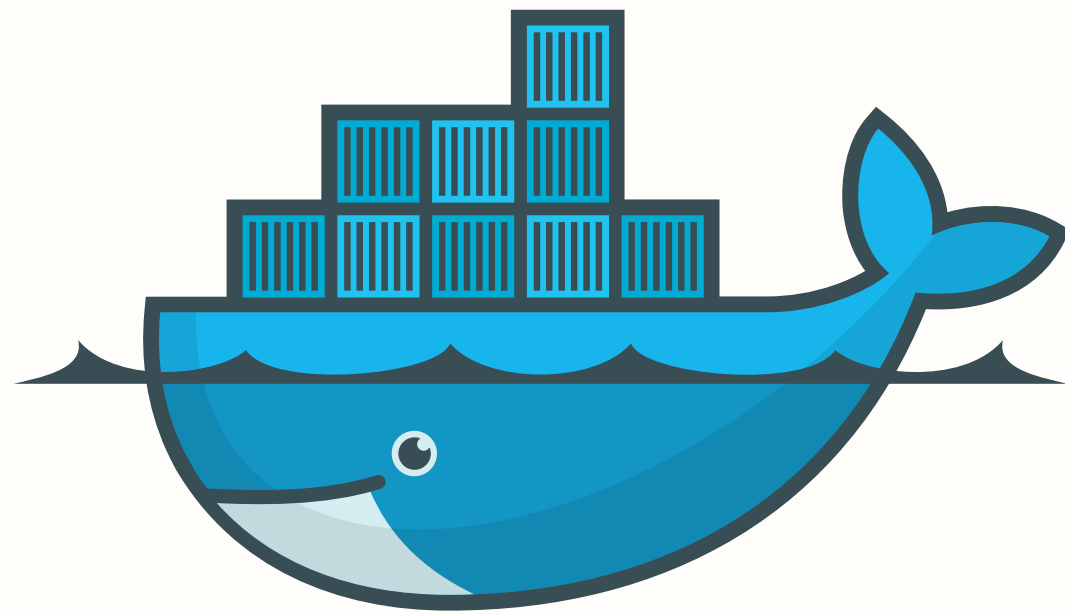
Problèmes
résolus par
Docker

Scalabilité

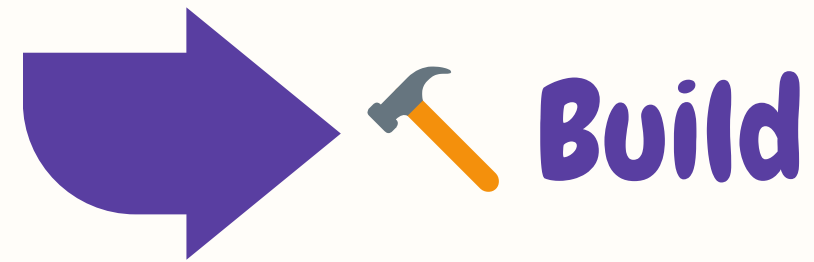
Conflits de dépendances



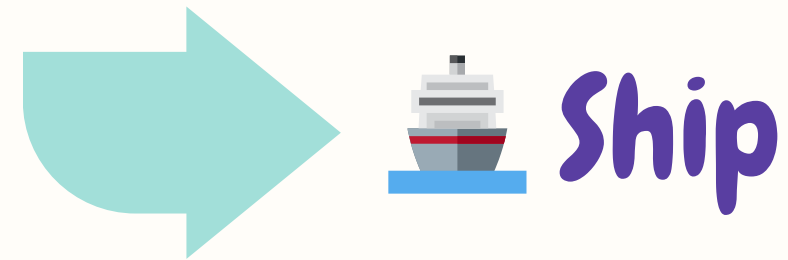
C'est quoi Docker ?



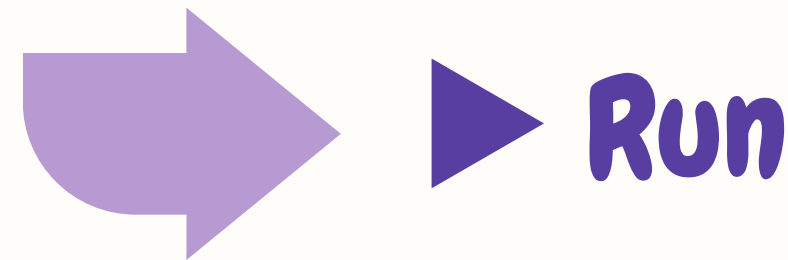
Principe de base de Docker :



Construire l'application



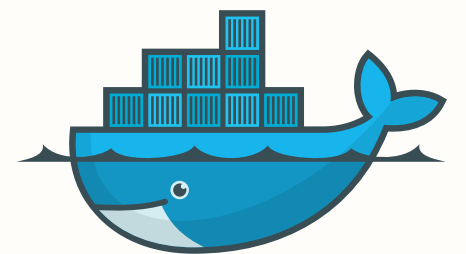
L'envoyer (dans une image Docker)



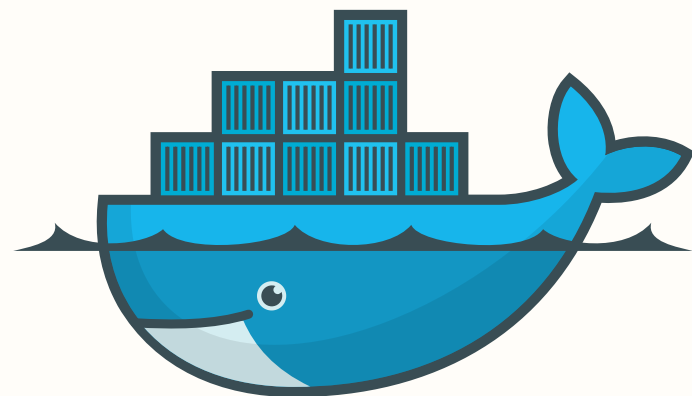
L'exécuter n'importe où



Docker est un outil qui permet de **créer, livrer (déployer)** et **exécuter** des applications facilement, peu importe ***l'environnement*** (Windows, Linux, Cloud, ...)



Comment fonctionne Docker ?

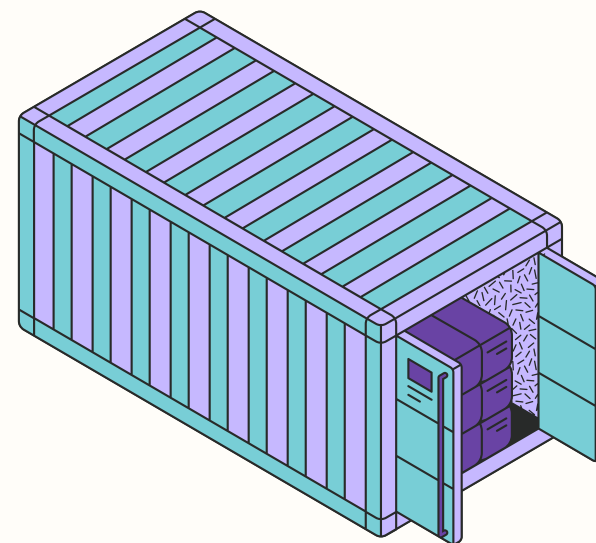


Comment fonctionne Docker ?



Docker repose sur une technologie appelée **conteneurs (containers)**. Ces conteneurs permettent d'**emballer une application avec tout ce dont elle a besoin** (*code, bibliothèques, configuration, dépendances, ...*) dans une seule **boîte isolée**.

Qu'est-ce qu'un conteneur Docker ?



Comment fonctionne Docker ?

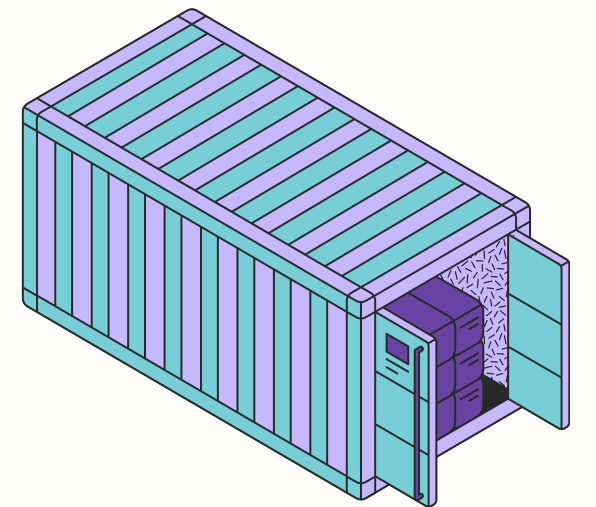


Docker repose sur une technologie appelée **conteneurs (containers)**. Ces conteneurs permettent d'**emballer une application avec tout ce dont elle a besoin** (*code, bibliothèques, configuration, dépendances, ...*) dans une seule **boîte isolée**.

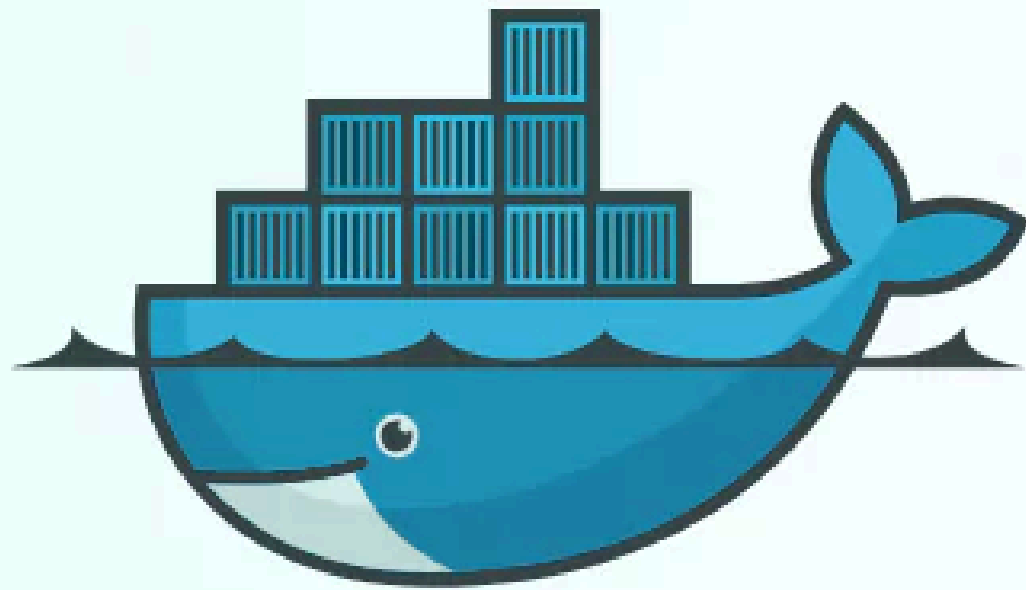
Qu'est-ce qu'un conteneur Docker ?



Un **conteneur Docker** est comme **une boîte virtuelle où ton application et toutes ses dépendances sont isolées** des autres applications et de l'environnement global. Cela garantit que ***l'application fonctionne de manière identique***, peu importe l'endroit où elle est exécutée.



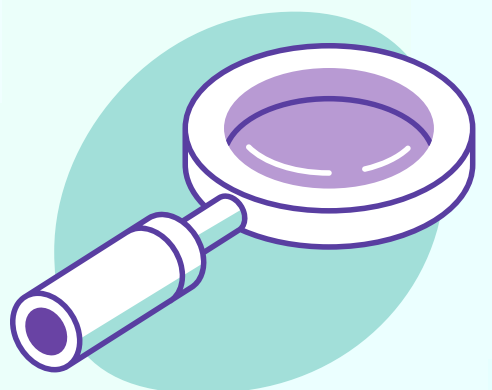
Quelle est la différence entre une image et un conteneur Docker ?



docker
Images



docker
Containers



Différence entre image et conteneur Docker

Image Docker :

Une image est **une photo statique** de l'état d'une application à un moment donné.
Elle contient tout le nécessaire pour exécuter l'application : le code, les bibliothèques, les dépendances et la configuration.
-->L'image est utilisée pour créer des conteneurs.



Pensez à une image comme un plan ou une recette pour créer un plat. L'image définit ce qu'il faut mettre dans le plat, mais ce n'est qu'un modèle, un modèle statique.



Différence entre image et conteneur Docker

Conteneur Docker :

Un conteneur est une instance en cours d'exécution d'une image Docker.

Lorsque tu lances une image, un conteneur est créé et exécute l'application.

--> Chaque **conteneur** est une application active fonctionnant dans un environnement isolé.



Un conteneur, c'est comme un plat préparé à partir de la recette (l'image). Une fois que le plat est préparé (le conteneur est lancé), il fonctionne de manière indépendante, en utilisant la recette (l'image).



Quelle est l'architecture de Docker ?

Docker repose sur quel modèle d'architecture ?

Quels sont les composants clés de l'architecture Docker ?

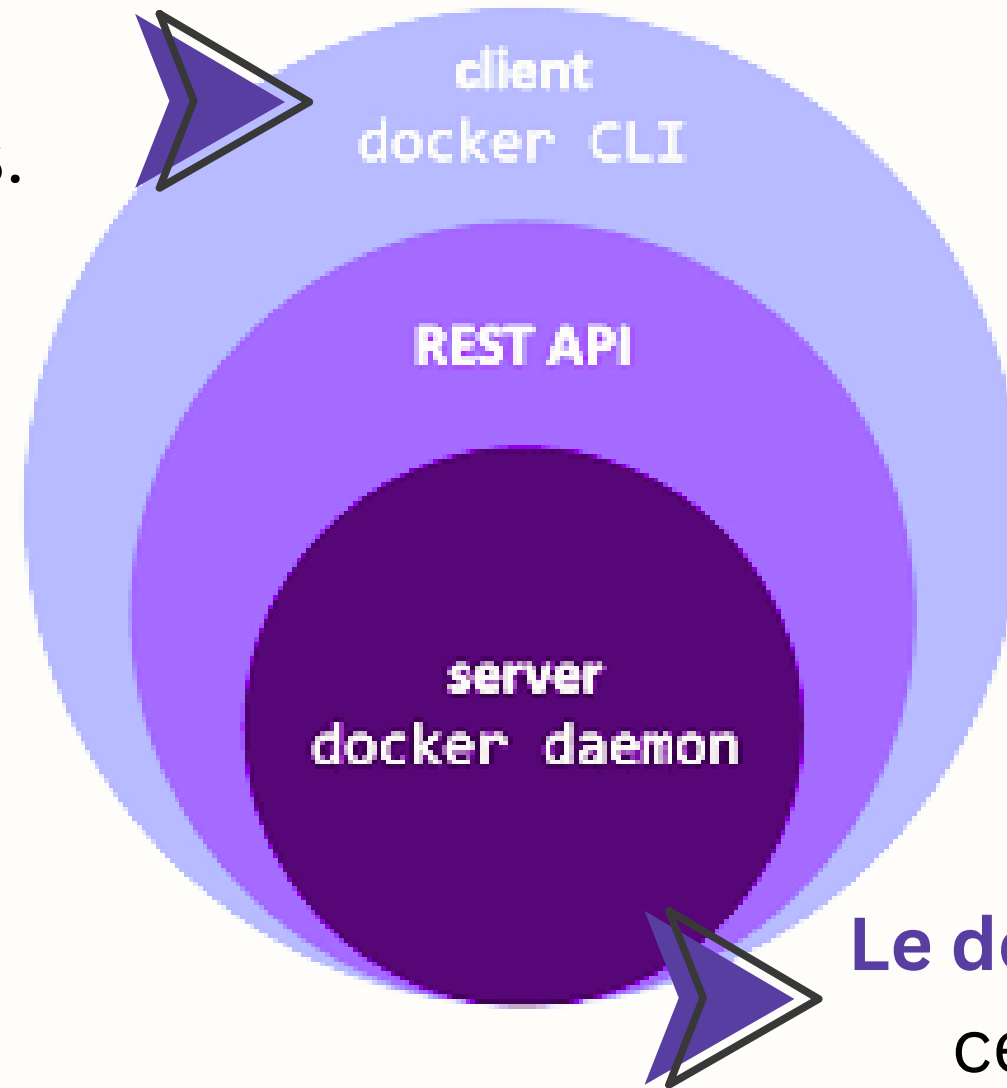
Comment chaque composant de l'architecture Docker interagit-il ?



Architecture de Docker

Docker fonctionne sur **une architecture client-serveur**, ce qui signifie qu'il y a **deux parties** principales qui communiquent :

Le client Docker :
celui qui tape les commandes.



💡 **Communication :**

Le client et **le démon** communiquent via une **API REST**, soit :

- à travers un socket UNIX (local)
- ou via le réseau (machine distante)

Le démon Docker :
celui qui exécute les actions en arrière-plan .

Les composants principaux de l'architecture Docker

Docker CLI c'est l'outil en ligne de commande qui permet de communiquer avec le moteur Docker (**Docker Engine**).

Il te permet de :

- Créer et lancer des conteneurs
- Gérer des images
- Supprimer, stopper ou visualiser les conteneurs
- Automatiser le déploiement d'applications



Les composants principaux de l'architecture Docker

Docker Daemon (dockerd) est le serveur qui gère ***les conteneurs***, ***les images***, et ***les volumes***. Il reçoit les commandes du client et les exécute.

Il peut également communiquer avec d'autres démons Docker sur différentes machines pour gérer **un cluster Docker**.

Le démon est responsable de l'exécution réelle des applications dans les conteneurs.



Les composants principaux de l'architecture Docker

Docker Registry est l'endroit où sont stockées les images Docker.

Docker Hub est un exemple de registre public, mais tu peux aussi avoir des registres privés.

Le registre te permet de télécharger des images (avec **docker pull**) et d'envoyer tes propres images (avec **docker push**).



Pourquoi Docker Hub est utile ?

Exemple pratique :



Sur **macOS**, **SQL Server** ne peut pas être installé *directement*, car **SQL Server** est principalement conçu pour **fonctionner sur Windows ou Linux**.



Pourquoi Docker Hub est utile ?

Exemple pratique :



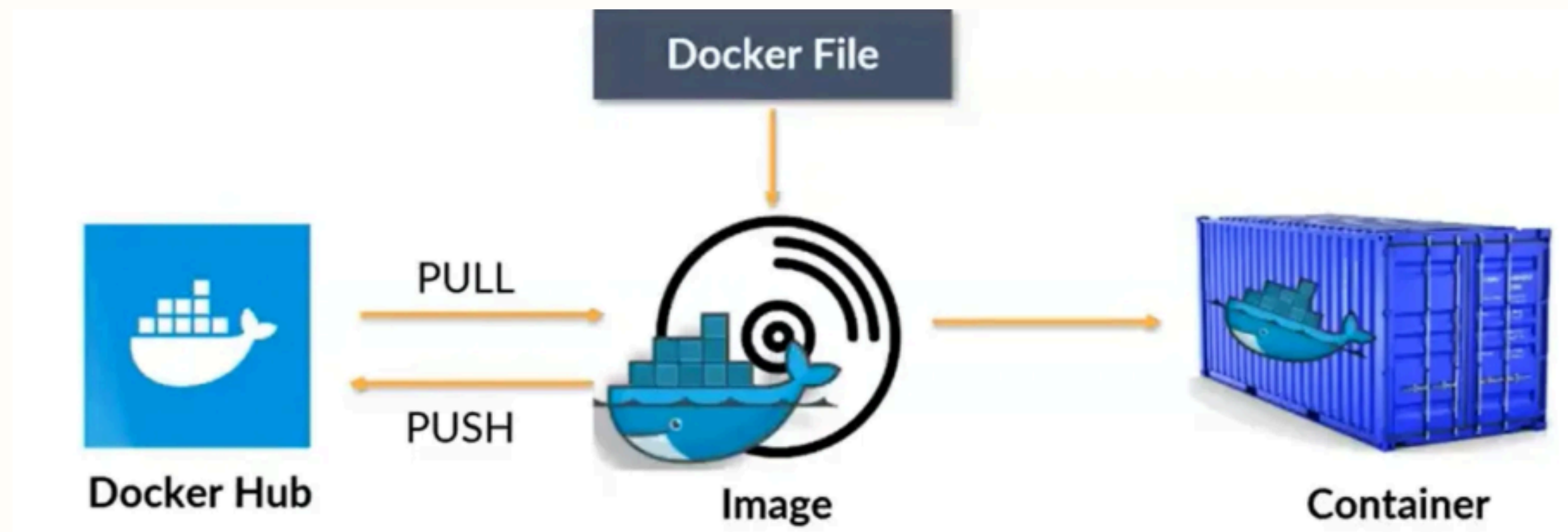
Recherche de l'image Docker

télécharger l'image

Exécution de l'image

Connexion à la base de données

Dockerisation d'une Application



**Quand vous créez une image Docker, est-ce
que vous devez tout inclure dans l'image,
même les fichiers inutiles ou temporaires ?**



Le fichier `.dockerignore`

Qu'est-ce qu'un fichier `.dockerignore` ?

Le fichier `.dockerignore` permet de spécifier **quels fichiers ou répertoires doivent être ignorés** lors *de la construction de l'[image Docker](#)*. Cela fonctionne de manière similaire à un fichier `.gitignore`.

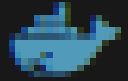
Pourquoi l'utiliser ?

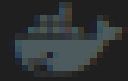
- **Réduction de la taille de l'image** : Exclure les fichiers inutiles réduit la taille de l'image.
- **Sécurité** : Éviter d'inclure des données sensibles (ex. : fichiers `.env`, clés privées, etc.).
- **Optimisation du processus** : Éviter d'inclure des fichiers temporaires ou des dépendances non nécessaires (`node_modules`, fichiers de logs,...).

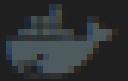


Le fichier .dockerignore

JS app.js

 Dockerfile 1 ●

 .dockerignore ●

 .dockerignore

1 node_modules/

2 *.log

3 .env

4 .git/

5 .vscode/

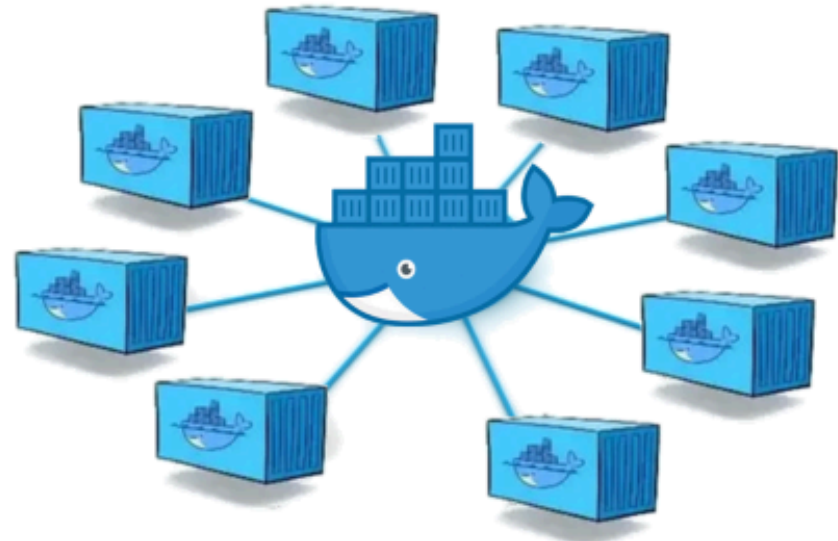


PARTIE 2

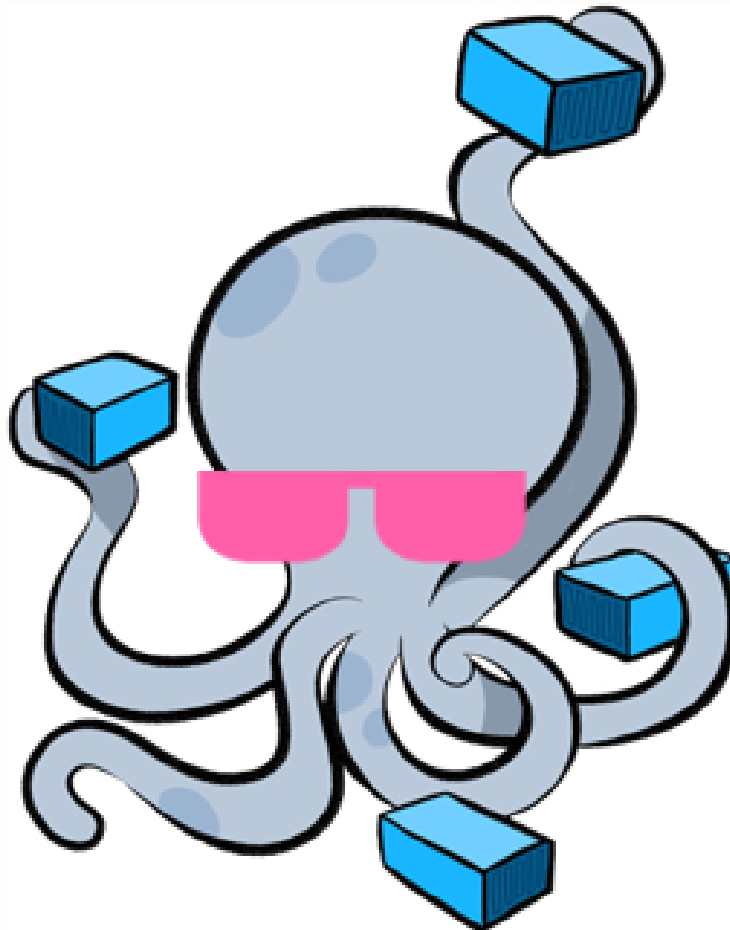
DOCKER DANS LE WORKFLOW DÉVELOPPEUR



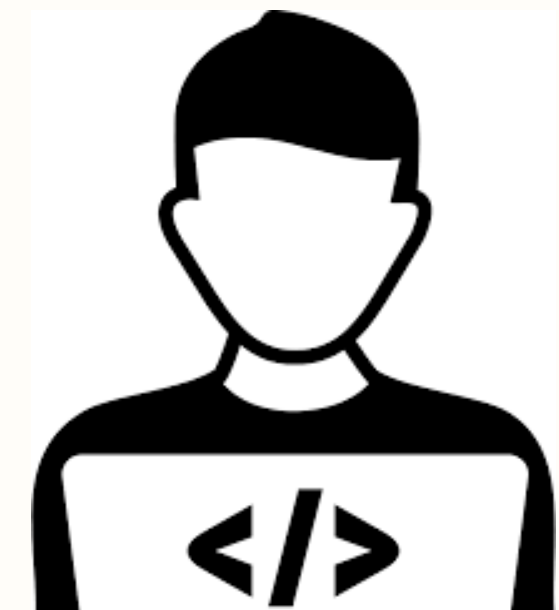
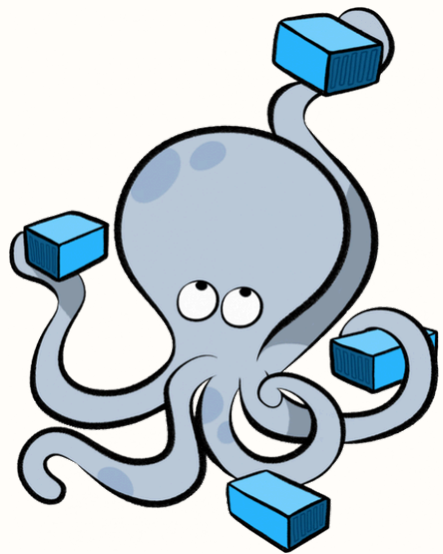
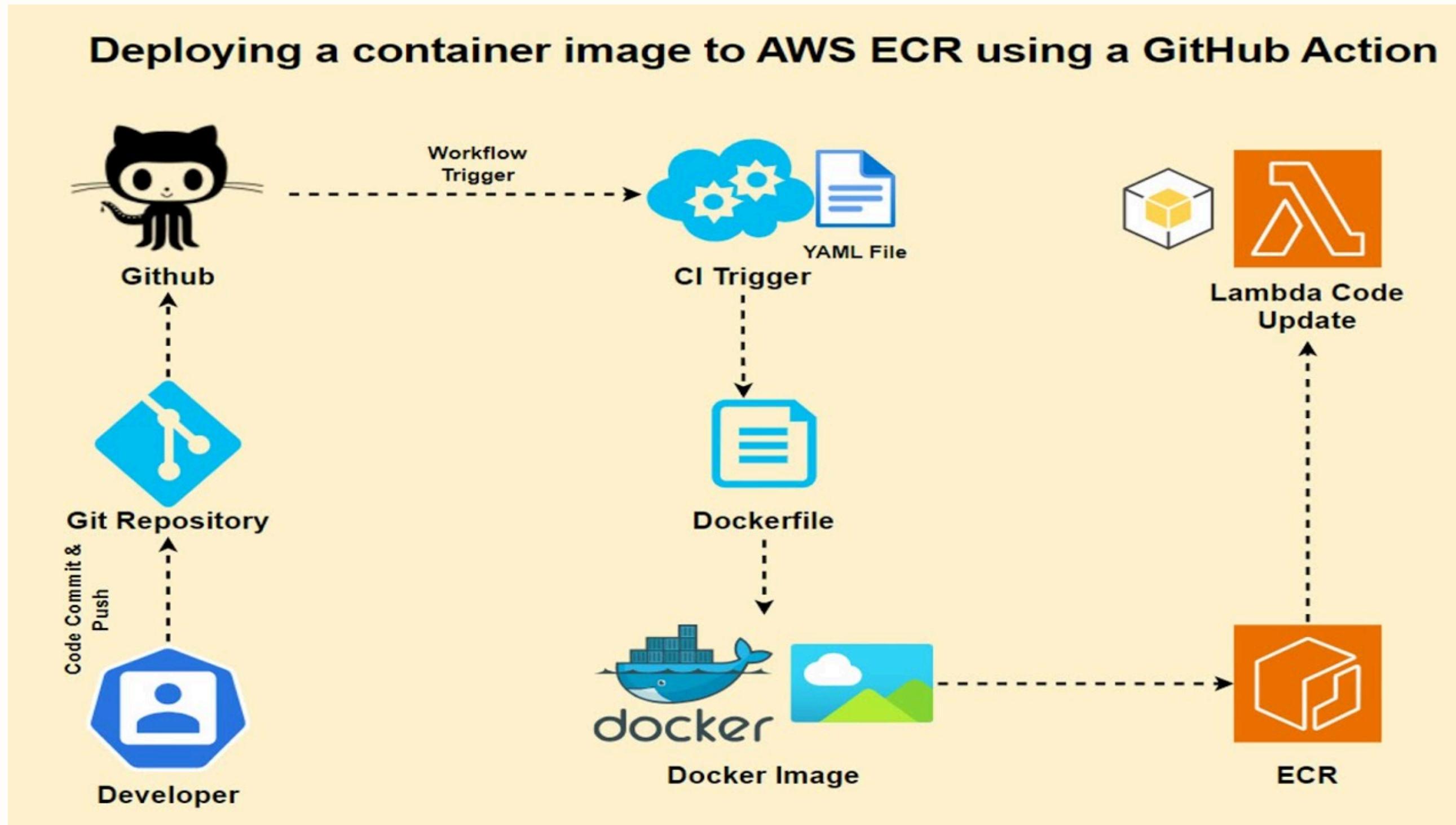
Docker en Profondeur



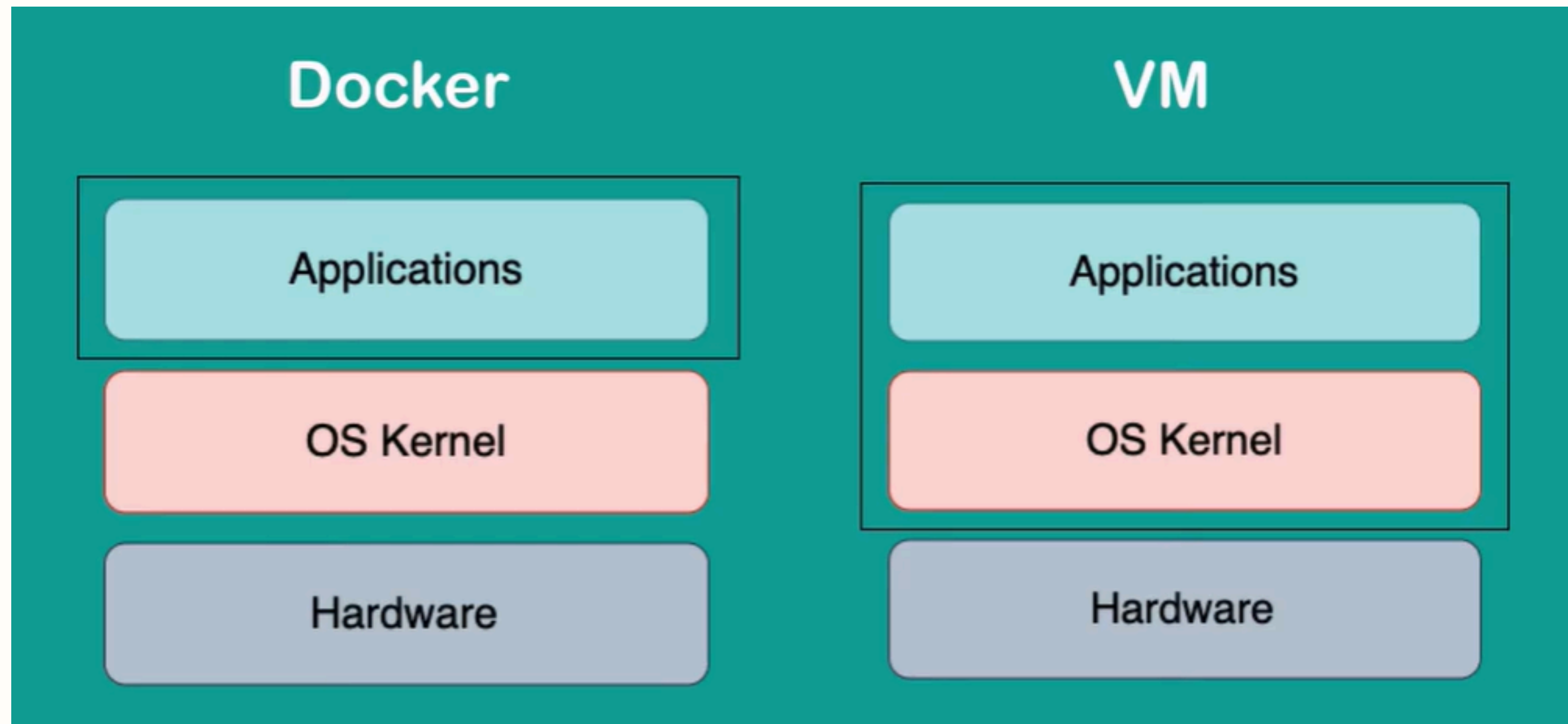
- Docker in the Dev Cycle
- Docker Compose
- Docker Network
- Docker Security



Docker dans le Cycle Dev



Docker Vs VM



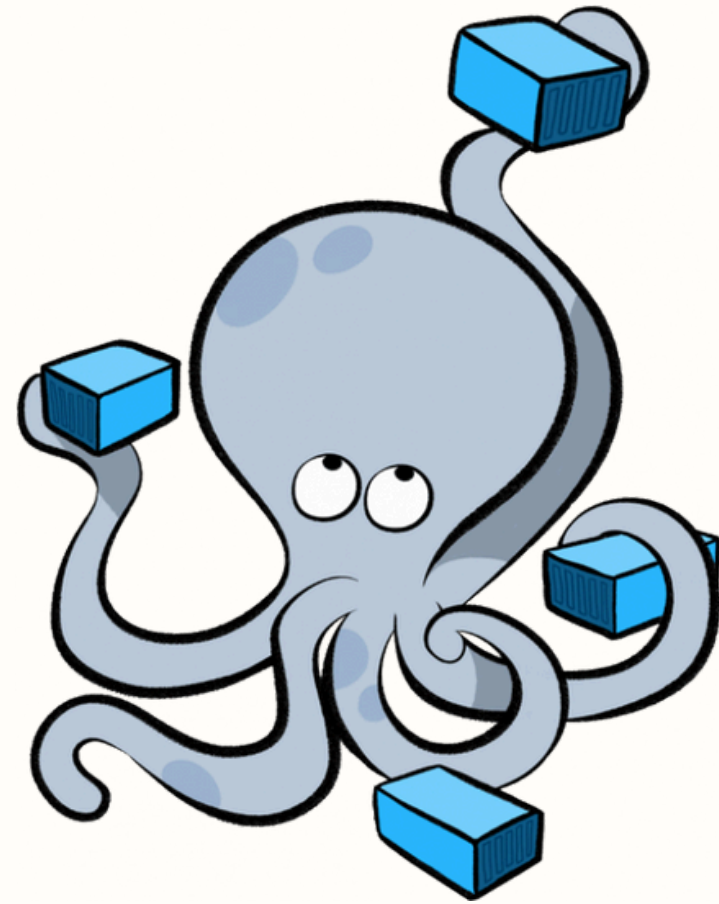
Docker (Conteneur)

- Partage le noyau du système hôte (pas d'OS complet par conteneur)
- Très léger et rapide
- Idéal pour exécuter des applications isolées

Machine Virtuelle (VM)

Contient un système d'exploitation complet (OS) par VM
Consomme plus de ressources (RAM, CPU, etc.)
Lente à démarrer, lourde à gérer

Pourquoi Docker Compose ?

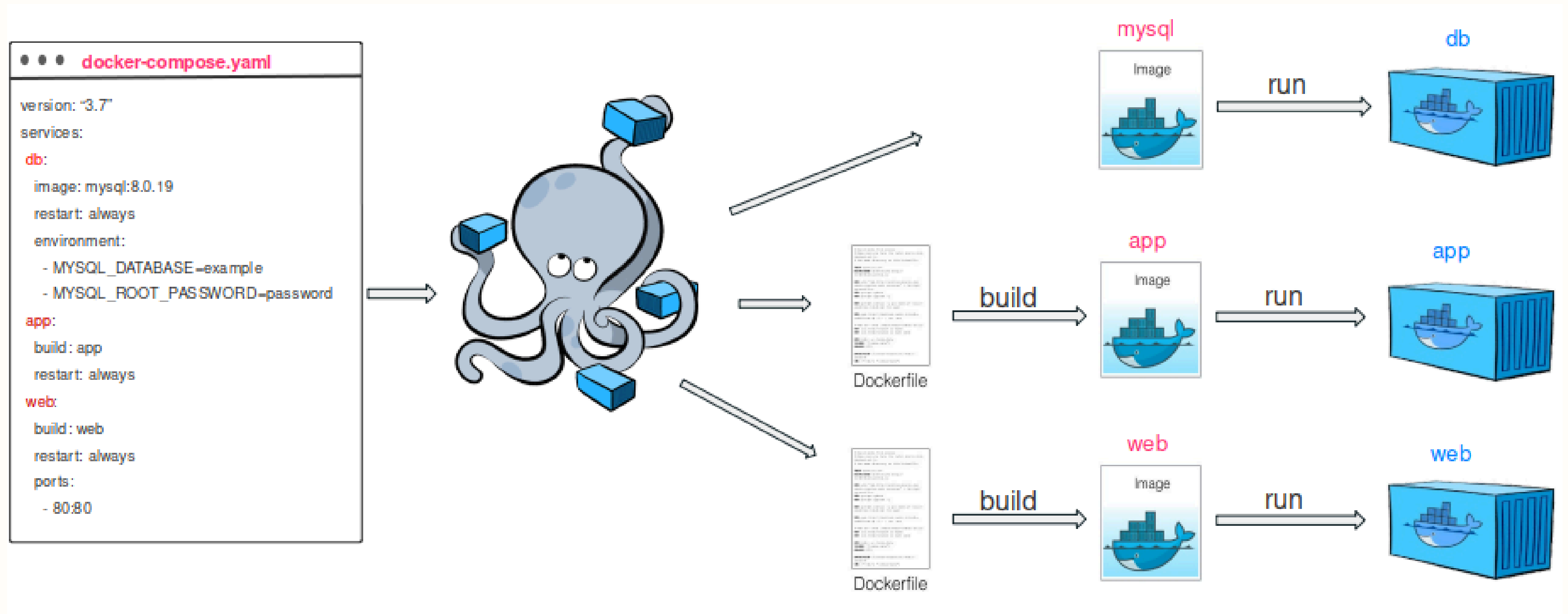


Lorsque ton application a plusieurs services (ex. **backend**, **base de données**, **frontend**), il devient :

- Fatigant de lancer chaque conteneur **à la main**
- Difficile de gérer les configurations et la **communication** entre services



Docker Compose



Exemple Docker Compose

/project-root

├── backend/

│ ├── Dockerfile [Dockerfile backend (Express.js)

├── frontend/

│ ├── Dockerfile [Dockerfile frontend (Vue.js)

└── docker-compose.yml [Orchestration globale (incluant PostgreSQL)]



Exemple de Docker Compose file

version: '3.8'

services:

backend:

build: ./backend

container_name: express-backend

restart: always

ports:

- "3000:3000"

environment:

- DB_HOST=db

- DB_USER=postgres

- DB_PASSWORD=mysecretpassword

- DB_NAME=mydb

depends_on:

- db

frontend:

build: ./frontend

container_name: vue-frontend

restart: always

ports:

- "8080:8080"

db:

image: postgres:14

container_name: postgres-db

restart: always

environment:

POSTGRES_USER: postgres

POSTGRES_PASSWORD:

mysecretpassword

POSTGRES_DB: mydb

volumes:

-

pgdata:/var/lib/postgresql/data

volumes:

pgdata:

Solution : Docker Compose

Un outil qui nous permet de :

- Définir tous les services de ton app dans un fichier docker-compose.yml
- Automatiser les **builds**, les **connexions**, les **ports**, les **variables**, etc.
- Lancer toute ton app avec une seule commande :

[docker-compose up](#)



Solution : Docker Compose

Un outil qui nous permet de :

- Définir tous les services de ton app dans un fichier docker-compose.yml
- Automatiser les **builds**, les **connexions**, les **ports**, les **variables**, etc.
- Lancer toute ton app avec une seule commande :

[docker-compose up](#)



Commandes essentielles

`docker-compose ps`

But : Voir les conteneurs lancés par Compose.

`docker-compose logs`

Affiche les logs de sortie (console) des services.

Tu peux cibler un service :

`docker-compose logs backend`

`docker-compose up -d`

But : Lancer tous les services définis dans le fichier `docker-compose.yml`.

- ♦ Option `-d` : "detached mode" → les conteneurs tournent en arrière-plan.
- ♦ Ce que ça fait concrètement :

Build des images si nécessaire (build:)

Création des conteneurs

Lancement des conteneurs connectés au réseau

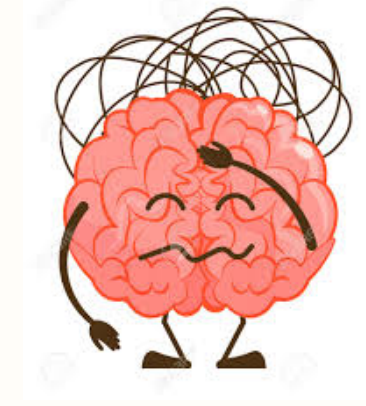
Affichage du mapping des ports

`docker-compose down`

But : Arrêter et supprimer tous les conteneurs, réseaux et volumes anonymes créés par `docker-compose up`.

- ♦ Ne supprime pas les volumes nommés (comme `pgdata`) à moins qu'on utilise `--volumes`.

Wait a minute!



- ? Comment mon conteneur backend sait où trouver la base de données ?
- ? Comment le frontend communique avec l'API backend ?



DOCKER NETWORK

Docker Compose a tout fait pour toi... mais tu ne sais pas encore ce qui relie réellement les services entre eux.

Et c'est à ce moment-là que tu te rends compte qu'il y a une brique fondamentale mais invisible :

DOCKER NETWORKING

HOST Network

- Le conteneur utilise directement le réseau de l'hôte
- Pas besoin de port mapping
- Pas d'isolation réseau (moins sécurisé)

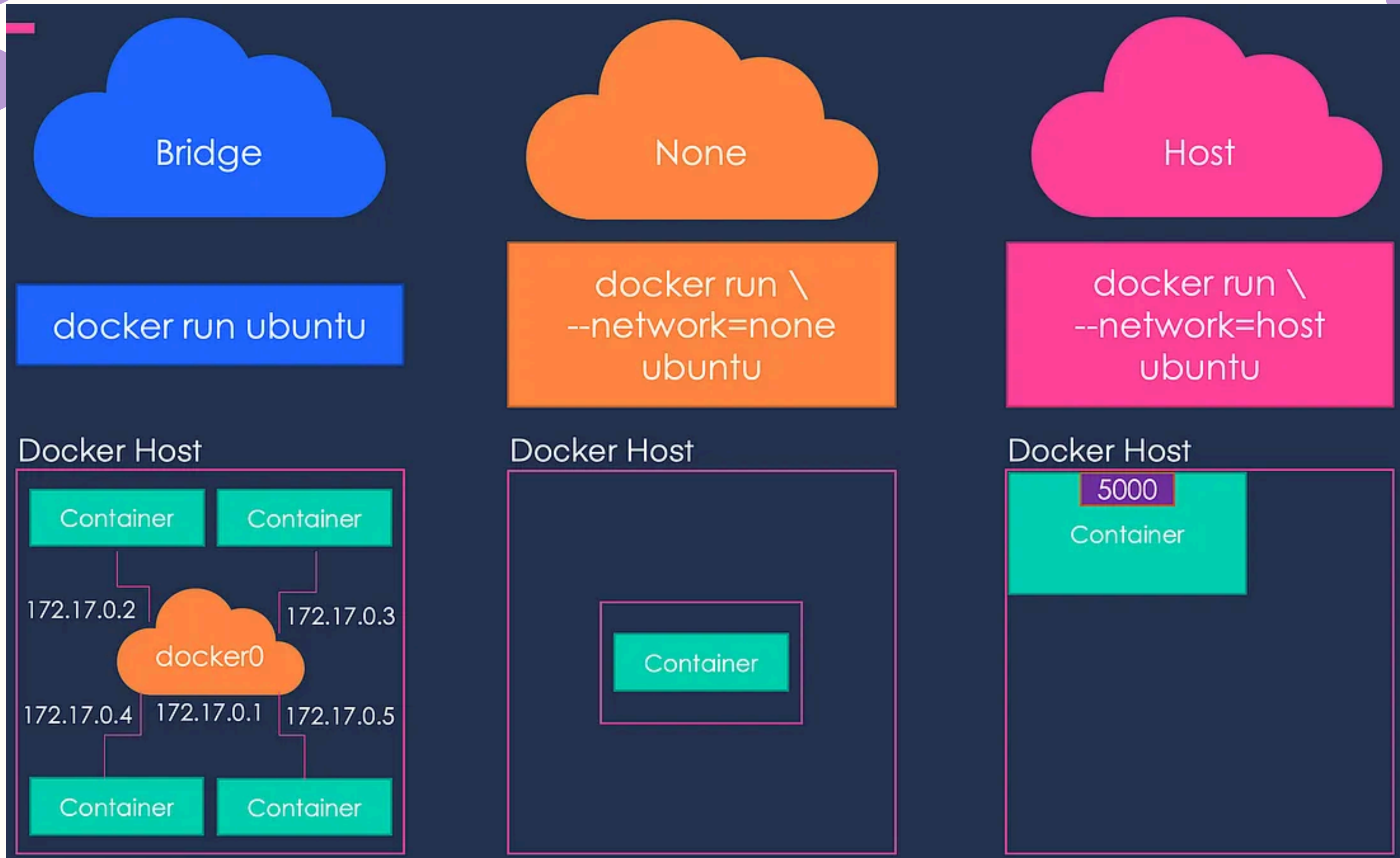
None Network

- Aucun accès réseau
- Le conteneur est totalement isolé
- Utilisé pour des tests ou du sandboxing

Bridge Network

- Conteneur a sa propre IP
- Peut communiquer avec d'autres conteneurs sur le même réseau
- Tu dois mapper les ports pour y accéder depuis l'extérieur (3000:3000)

DOCKER NETWORK



[BRIDGE MODE] Port Mapping ??

Quand tu fais tourner un conteneur Docker, il est isolé dans son propre réseau.

Donc ton application tourne à l'intérieur du conteneur, mais n'est pas directement accessible depuis ta machine.

👉 Le port mapping permet de connecter un port de ta machine (host) à un port dans le conteneur.

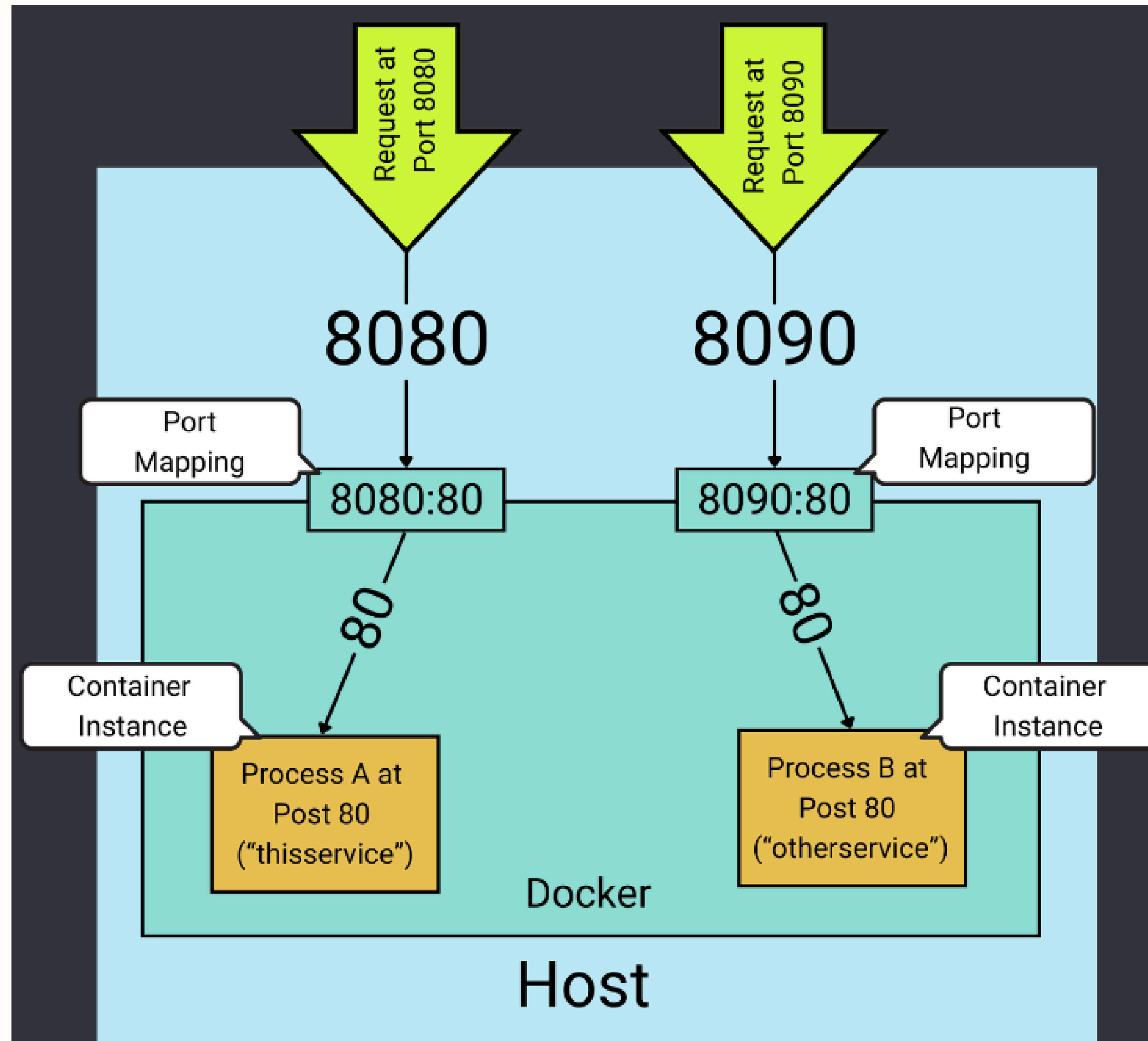
-p <port_host>:<port_conteneur>

<port_conteneur>

C'est le port sur lequel ton application écoute dans le conteneur

<port_host>

C'est le port de ta machine (PC) sur lequel tu veux accéder à l'app



Problème naturel en bridge

Ton conteneur backend
(Express.js) écoute sur 3000

Il est dans le réseau bridge

Tu veux y accéder depuis ton
navigateur ou Postman

Mais :

✗ Sans port mapping, impossible
d'y accéder depuis localhost car
ton navigateur ne peut pas
atteindre directement l'IP du
conteneur.

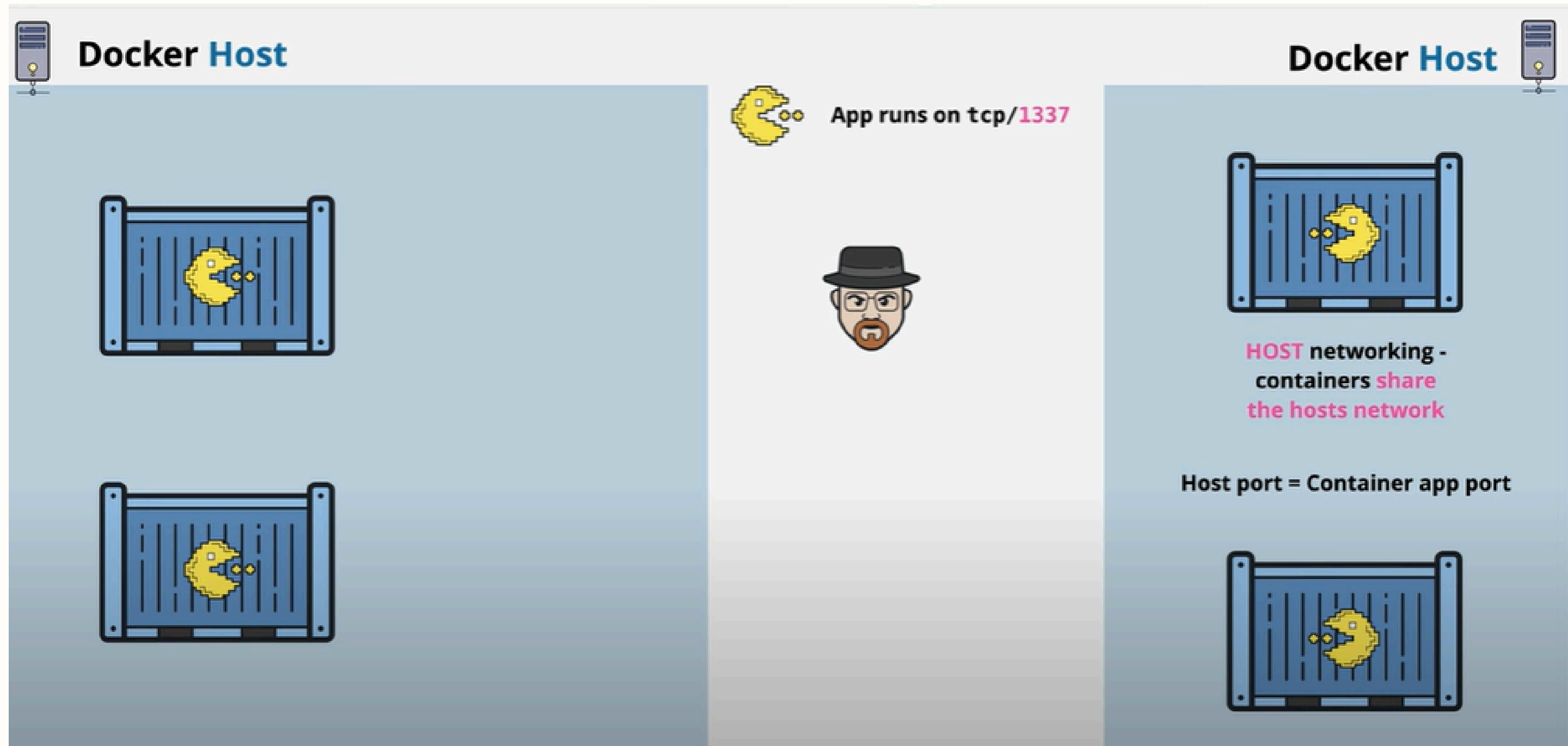
Solution : Port mapping

```
docker run -d --name backend \  
  --network bridge  
  -p 8080:3000 \backend-image
```

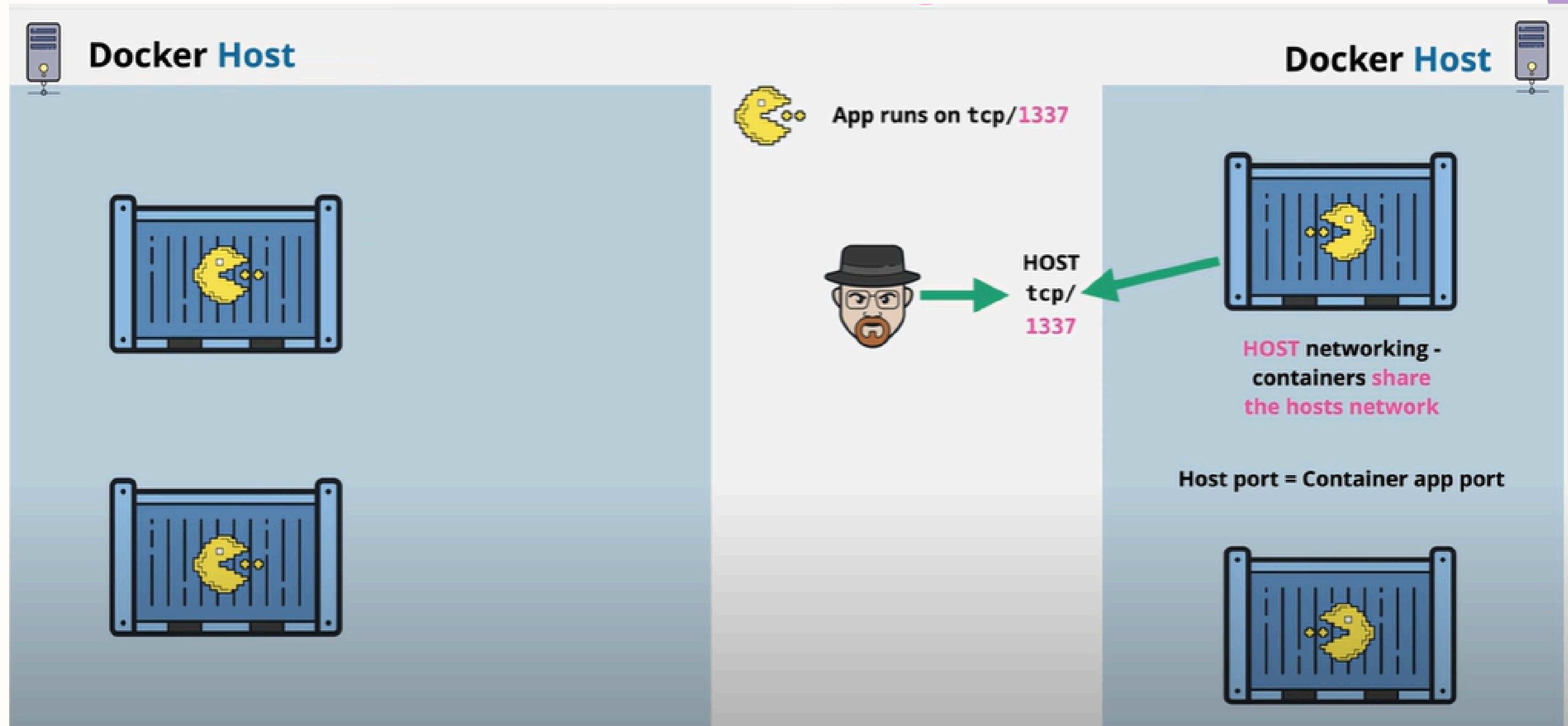
3000 --> Port dans le conteneur
(où ton Express écoute)

8080 --> Port de ta machine
(l'hôte) sur lequel tu accèdes à
l'application
-p 8080:3000

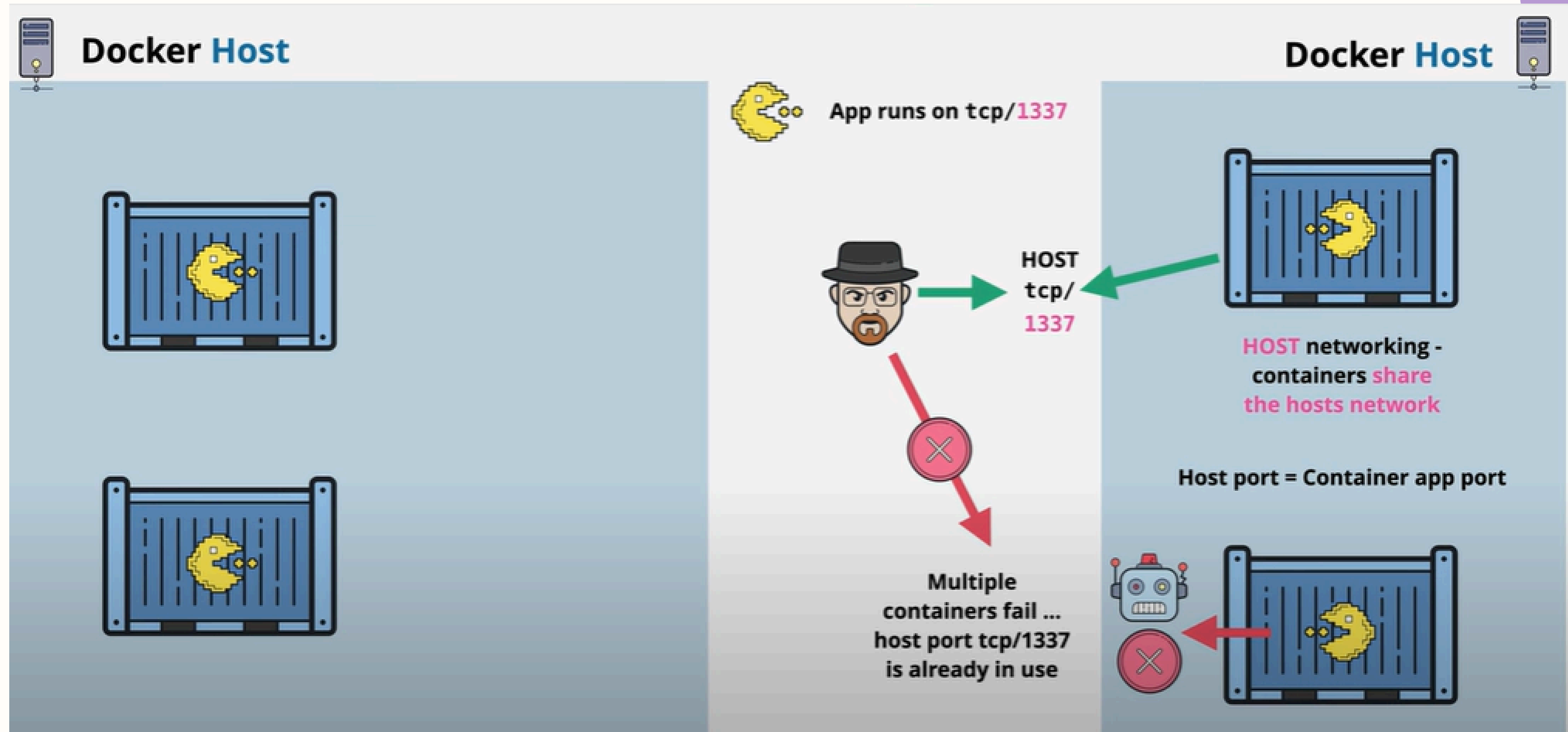
Docker Volumes-exemple



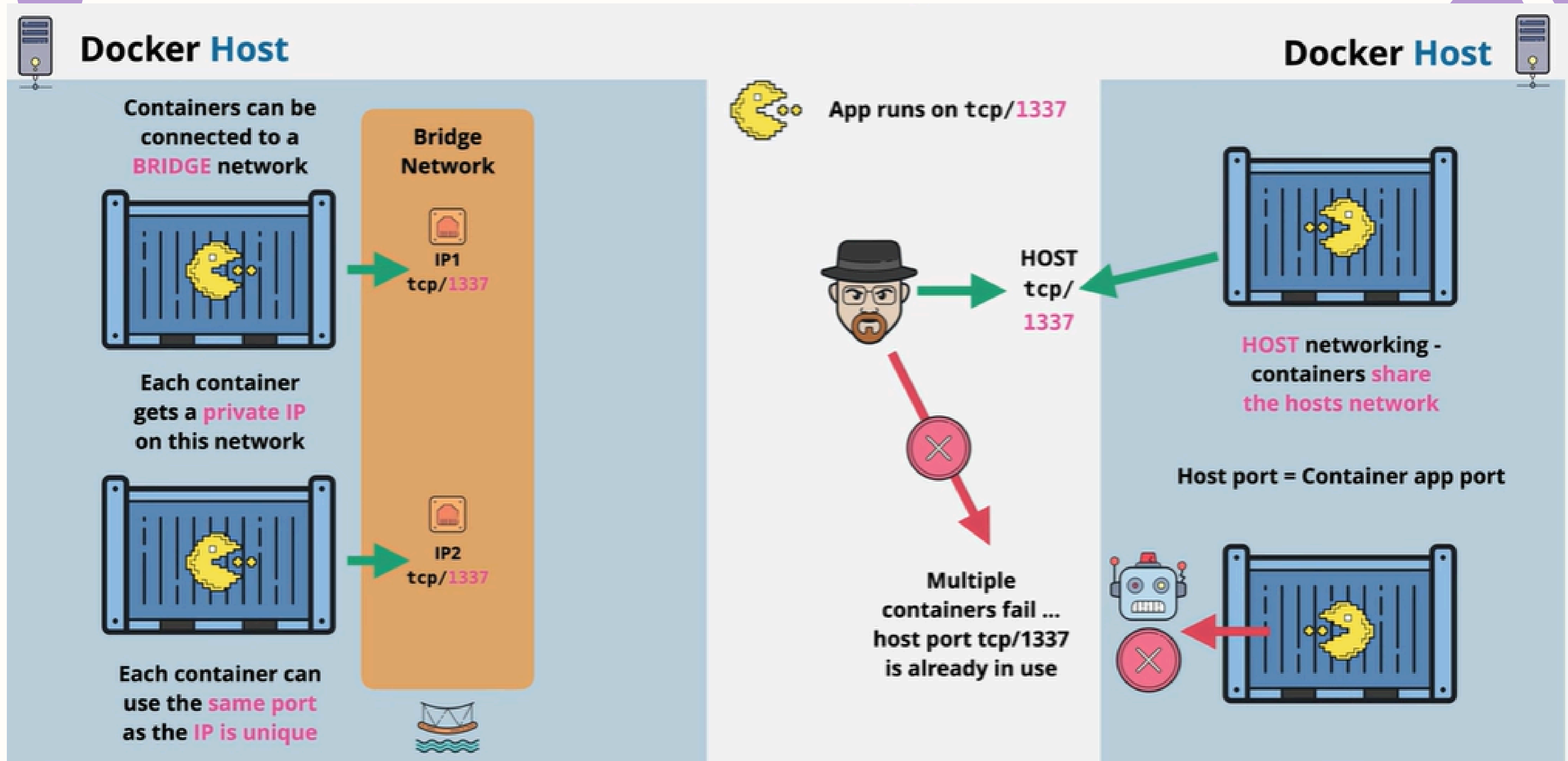
Docker Volumes-example



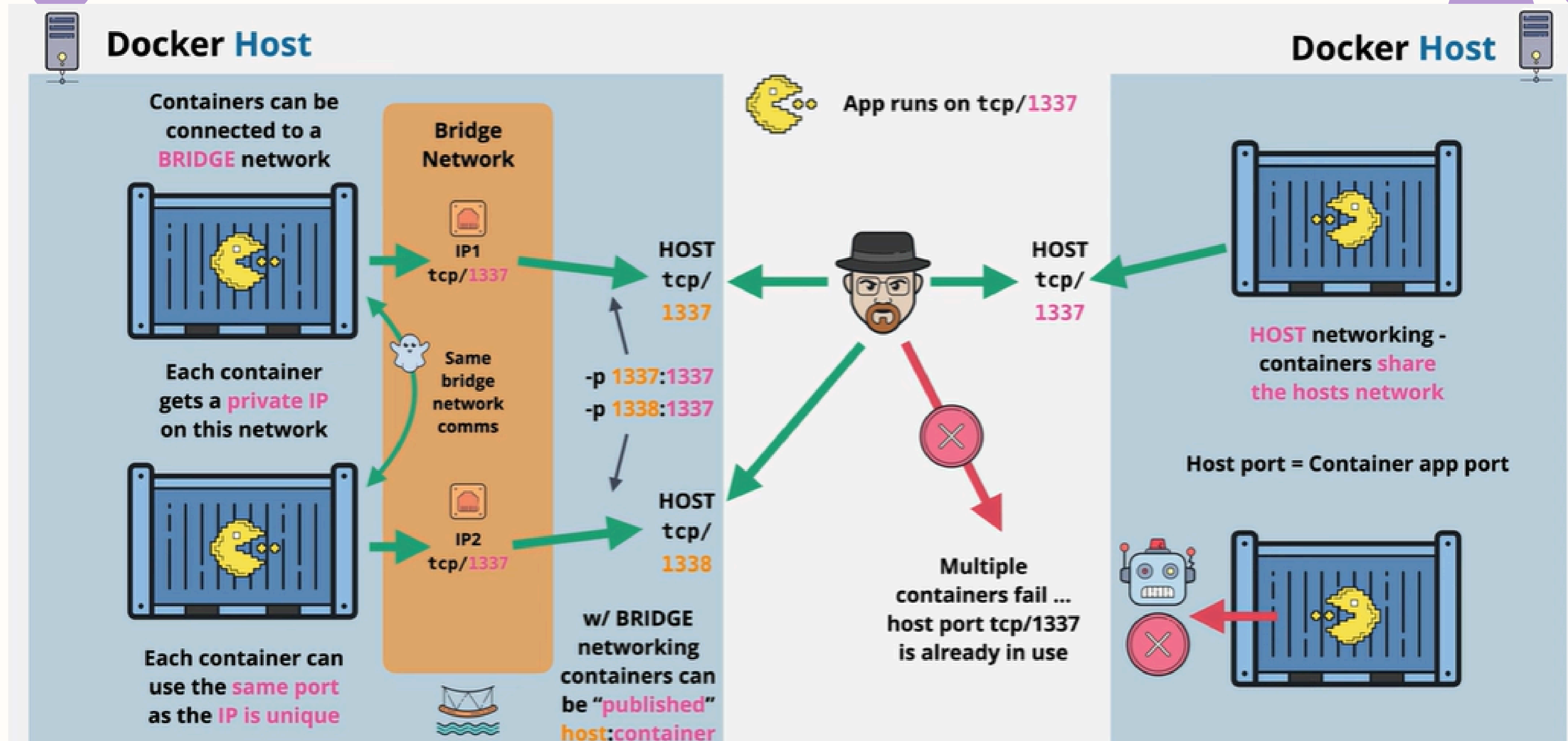
Docker Volumes-exemple



Docker Volumes-example



Docker Volumes-example



Docker Volumes

Pourquoi a-t-on besoin des volumes ?

Par défaut, les données à l'intérieur d'un conteneur sont éphémères.

Si tu fais :

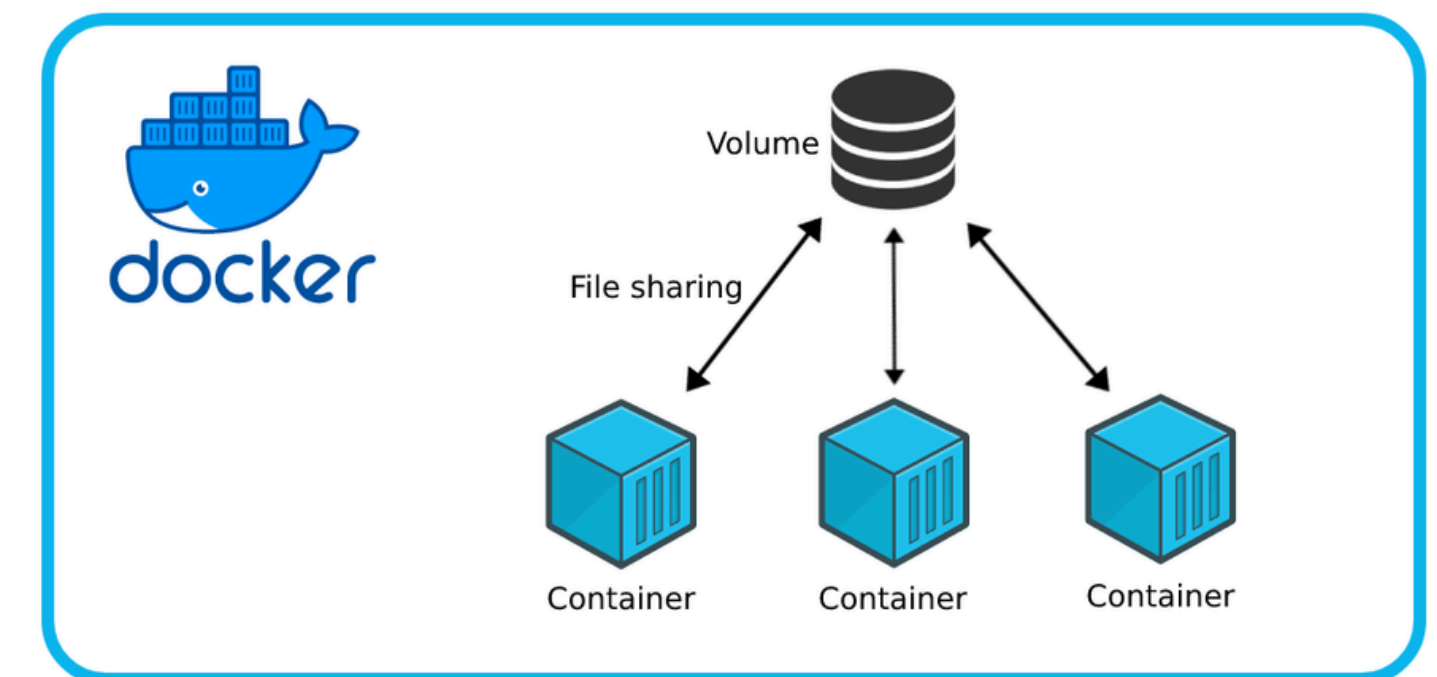
`docker run -d postgres`

Dès que tu supprimes ce conteneur
(`docker rm`), tout son contenu est perdu !!



Problématique :

“Je veux que ma base de données garde ses données, même si je supprime ou redémarre le conteneur.”



Solution : les Docker Volumes

Un volume est une zone de stockage permanente gérée par Docker, indépendante du cycle de vie du conteneur.

Création d'un volume

Manuellement :

```
docker volume create monvolume
```

Automatiquement
(via docker run) :

```
docker run -d \  
-v monvolume:/app/data \  
myapp
```

Que fait Docker ici ?

1. Crée un dossier spécial sur ton disque dur
2. Monte ce dossier dans le conteneur au chemin /app/data
3. Tout ce que ton conteneur écrit ici est sauvegardé dans le volume
4. Même si tu supprimes le conteneur → les données restent.



DOCKER SECURITY BEST PRACTICES

| HOW TO SECURE YOUR CONTAINER.



DOCKER SECURITY

PROBLÉMATIQUE

“DOCKER ME PERMET DE LANCER DES CONTENEURS TRÈS FACILEMENT,
MAIS EST-CE QUE CES CONTENEURS SONT VRAIMENT ISOLÉS ?
EST-CE QUE QUELQU’UN POURRAIT ACCÉDER À MA MACHINE VIA UN
CONTENEUR ?
ET QUE FAIRE POUR ÉVITER DES FAILLES DANS MES IMAGES ?”

BEST PRACTICES

1. Prefer lightweight base image
2. Update base image
3. Run container as non-root user
4. Resource quotas

UTILISE DES IMAGES DE BASE LÉGÈRES (AVEC LE MINIMUM D'OUTILS)

? POURQUOI ?

MOINS IL Y A D'OUTILS → MOINS IL Y A DE FAILLES POTENTIELLES
PLUS RAPIDE À TÉLÉCHARGER, PLUS FACILE À MAINTENIR

TOUJOURS GARDER TON IMAGE DE BASE À JOUR

? POURQUOI ?

LES FAILLES DE SÉCURITÉ SONT SOUVENT CORRIGÉES DANS LES
NOUVELLES VERSIONS

UNE VIEILLE IMAGE = VULNÉRABILITÉS NON CORRIGÉES 😬

NE JAMAIS EXÉCUTER L'APPLICATION EN TANT QUE ROOT DANS LE
CONTENEUR

? POURQUOI ?

SI UN PIRATE S'ÉCHAPPE DU CONTENEUR → IL DEVIENT ROOT SUR TA
MACHINE

TRÈS DANGEREUX EN PRODUCTION

LIMITE LES RESSOURCES UTILISÉES PAR UN CONTENEUR (MÉMOIRE, CPU)

? POURQUOI ?

- POUR ÉVITER QU'UN CONTENEUR NE PLANTE TOUTE LA MACHINE
- POUR MIEUX CONTRÔLER LES USAGES ET PRÉVENIR LES ATTAQUES DE
TYPE "DOS INTERNE"





PARTIE 3

DOCKER & CI/CD

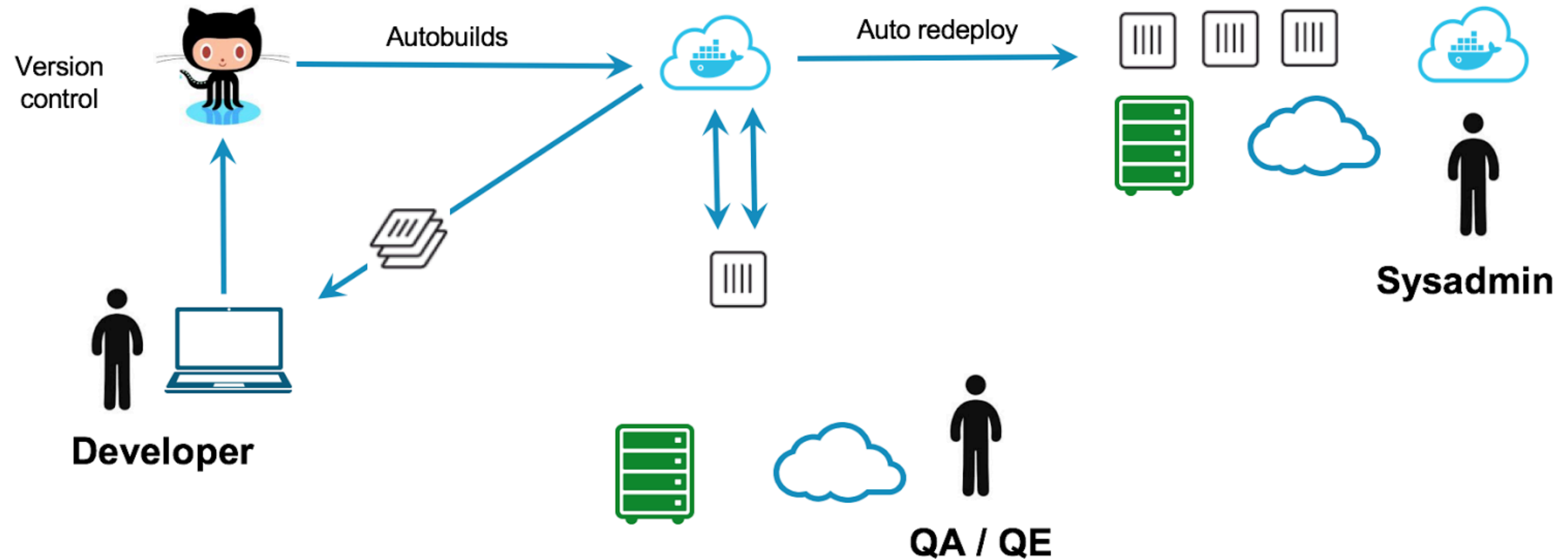
DANS UN PROJET RÉEL

**OBJECTIF : ÉLEVER LA VISION VERS DES
USAGES PROFESSIONNELS**



Docker dans un pipeline CI/CD

1. Development > 2. Test > 3. Stage / Production



Plan

Code

Build

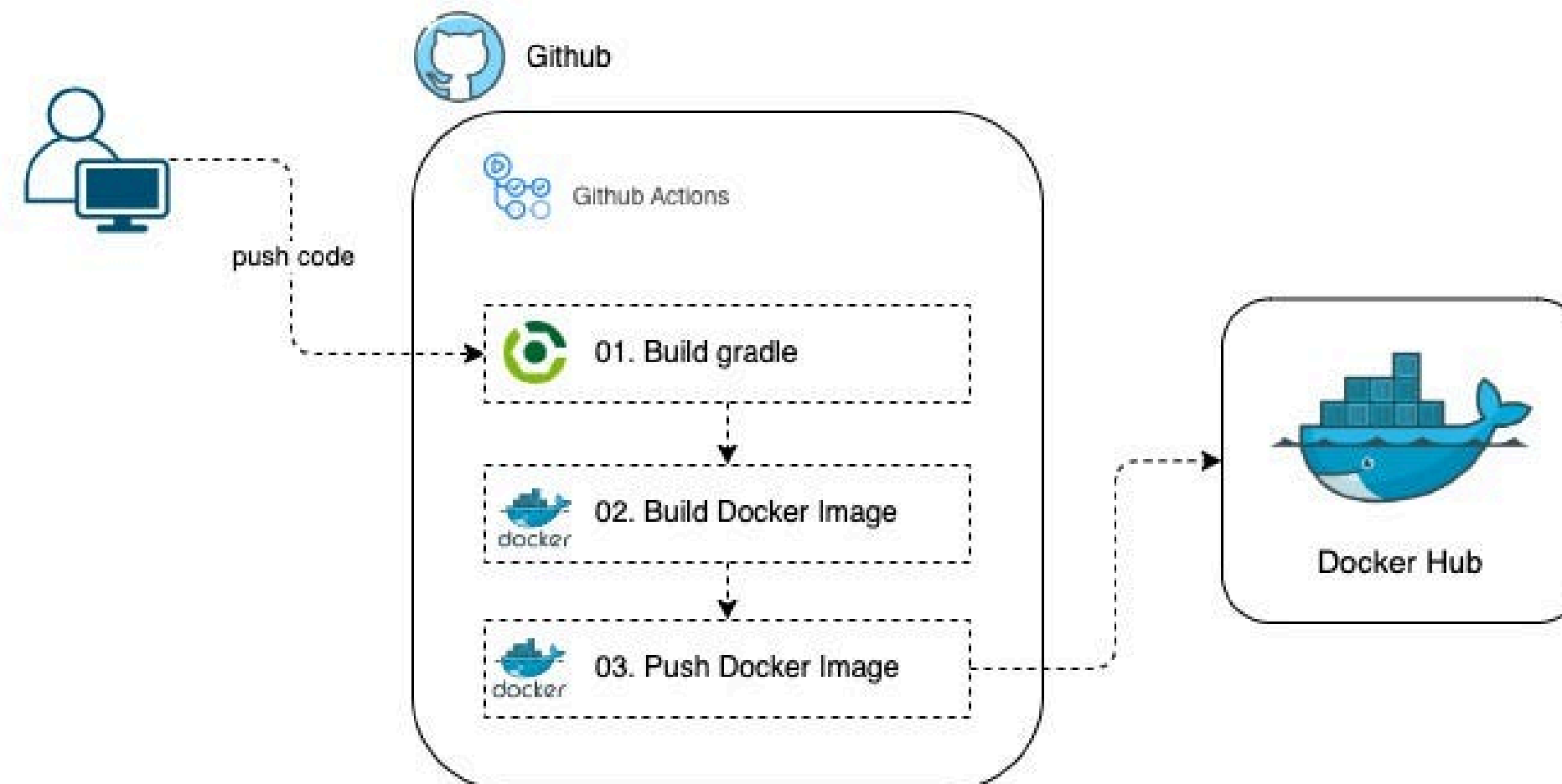
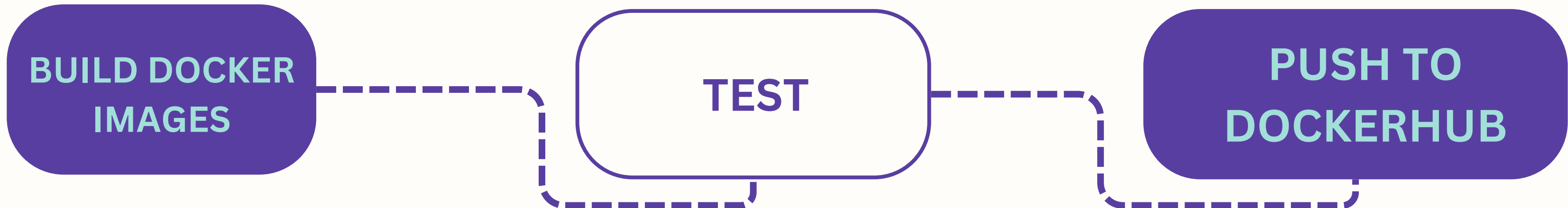
Test

Release

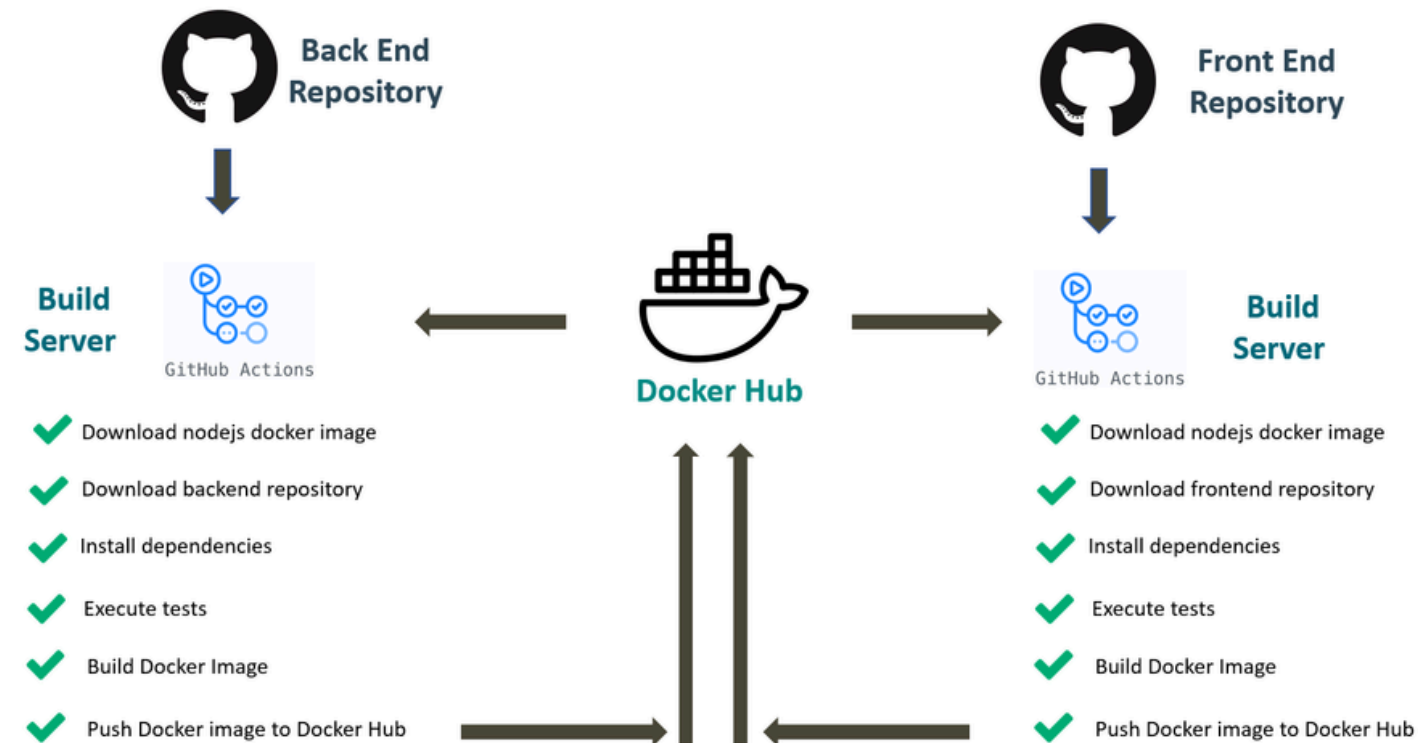
Deploy

Operate

CI AVEC DOCKER



CI AVEC DOCKER



```
name: CI/CD Pipeline

on:
  push:
    branches:
      - main # You can adjust the branch if needed, like 'master' or any other branch
  pull_request:
    branches:
      - main

jobs:
  build-and-deploy:
    runs-on: ubuntu-latest
    steps:
      # Step 1: Checkout the repository
      - name: Checkout code
        uses: actions/checkout@v2

      # Step 2: Set up Docker Buildx (for multi-platform support)
      - name: Set up Docker Buildx
        uses: docker/setup-buildx-action@v1

      # Step 3: Cache Docker layers to speed up subsequent builds
      - name: Cache Docker layers
        uses: actions/cache@v2
        with:
          path: /tmp/.buildx-cache
          key: ${{ runner.os }}-buildx-${{ github.sha }}
          restore-keys: |
            ${{ runner.os }}-buildx-
```

```
# Step 4: Backend Docker build and test
- name: Build Backend Docker image
  working-directory: ./backend # Replace with the correct path to your backend
  run: |
    docker build -t username/backend-image:${{ github.sha }} .

- name: Run Backend Tests
  working-directory: ./backend # Replace with the correct path to your backend
  run: |
    docker run username/backend-image:${{ github.sha }} npm test # Ensure you have

# Step 5: Frontend Docker build and test
- name: Build Frontend Docker image
  working-directory: ./frontend # Replace with the correct path to your frontend
  run: |
    docker build -t username/frontend-image:${{ github.sha }} .

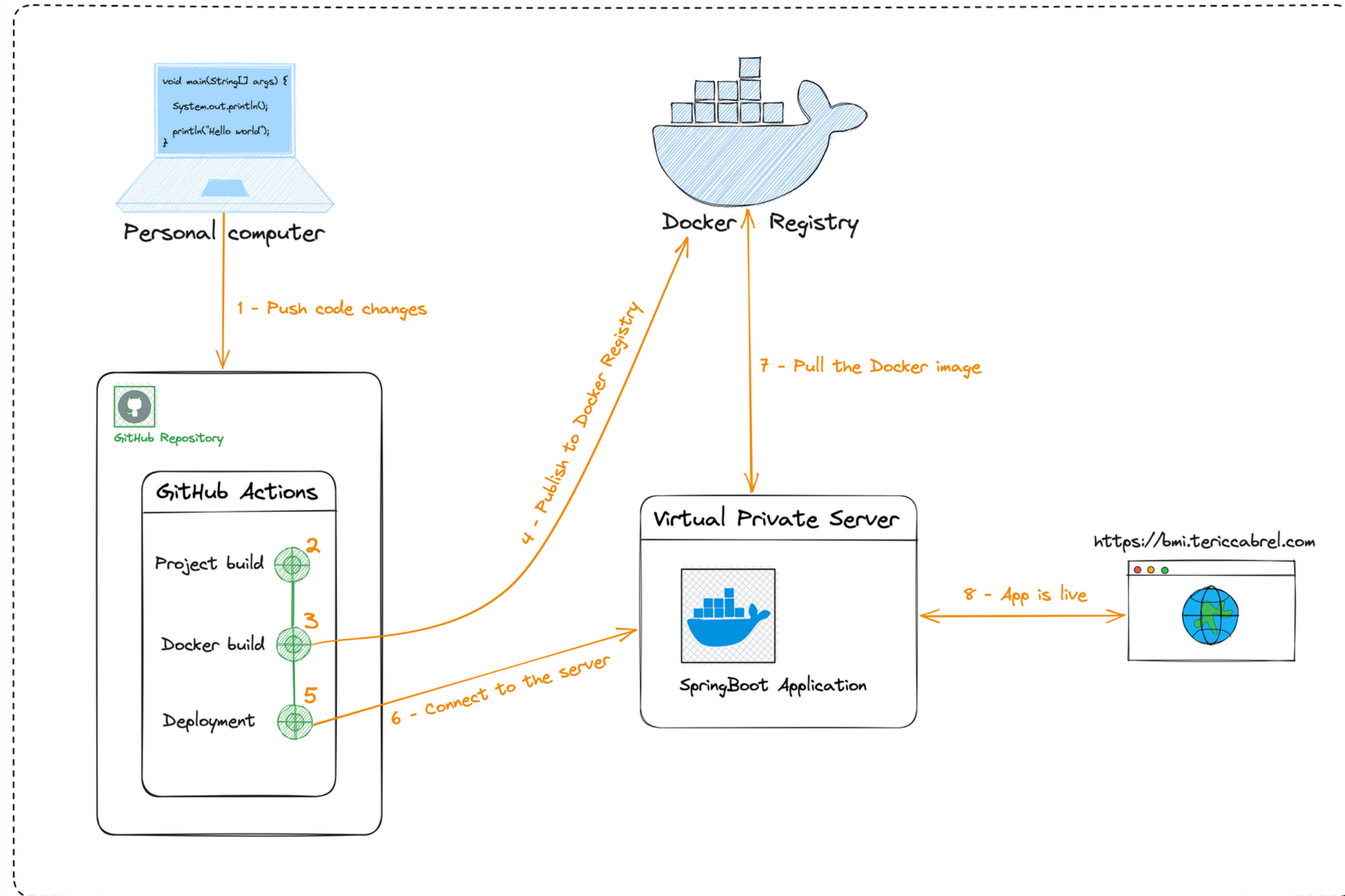
- name: Run Frontend Tests
  working-directory: ./frontend # Replace with the correct path to your frontend
  run: |
    docker run username/frontend-image:${{ github.sha }} npm run test # Ensure you

# Step 6: Log in to Docker Hub
- name: Log in to Docker Hub
  uses: docker/login-action@v2
  with:
    username: ${{ secrets.DOCKER_USERNAME }}
    password: ${{ secrets.DOCKER_PASSWORD }}

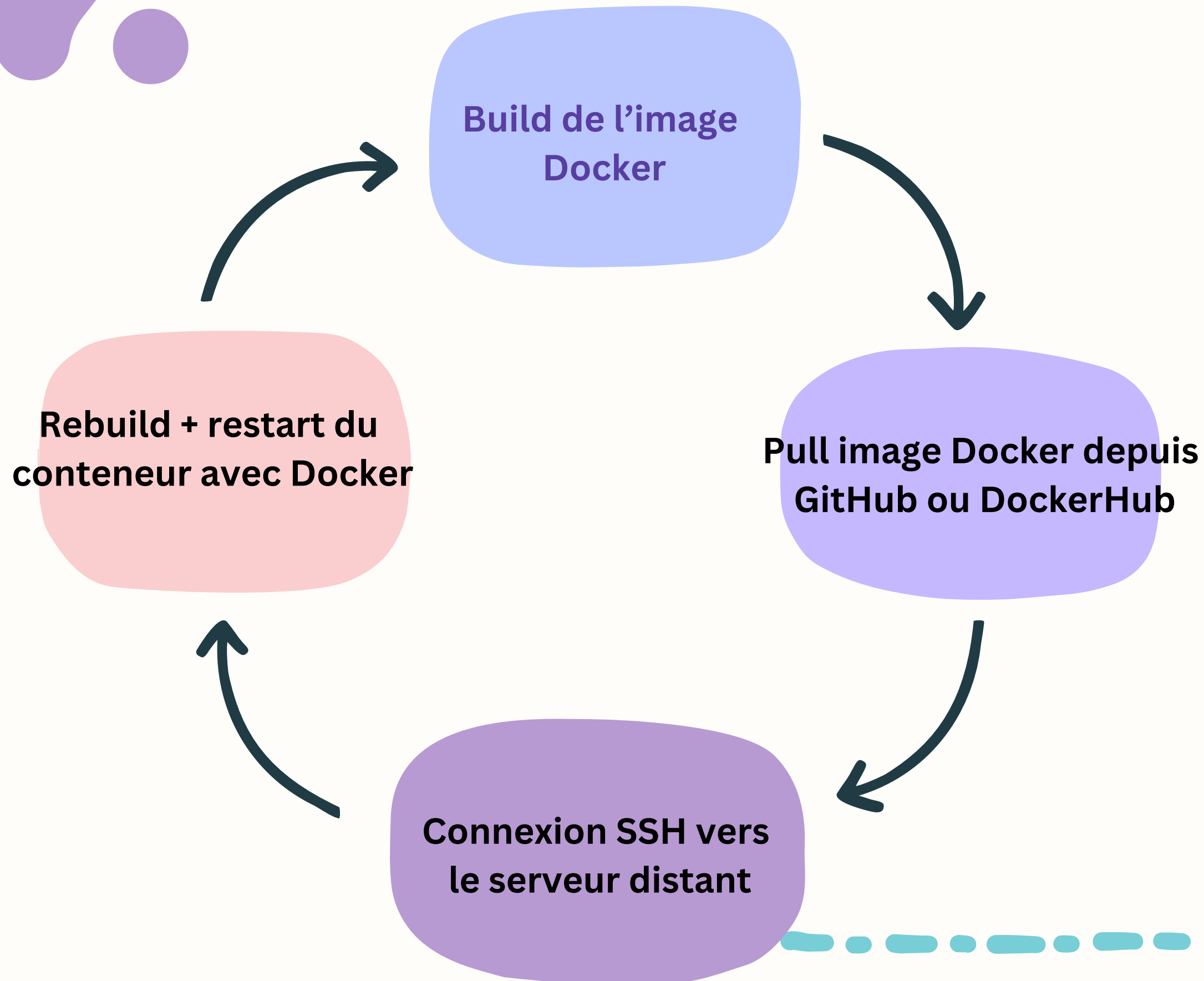
# Step 7: Push Backend Docker image to Docker Hub
- name: Push Backend Docker image to Docker Hub
  run: |
    docker push username/backend-image:${{ github.sha }}

# Step 8: Push Frontend Docker image to Docker Hub
- name: Push Frontend Docker image to Docker Hub
  run: |
    docker push username/frontend-image:${{ github.sha }}
```


CD WITH DOCKER



CD WITH DOCKER



```
name: CD Pipeline with Docker and Vercel

# Trigger this action on push events to the 'main' branch
on:
  push:
    branches:
      - main # The workflow runs on every push to the 'main' branch

jobs:
  deploy:
    runs-on: ubuntu-latest # The job will run on the latest Ubuntu runner

    steps:
      # Step 1: Checkout the code from the GitHub repository
      - name: Checkout repository
        uses: actions/checkout@v2 # This step checks out the code so we can access it

      # Step 2: Log in to Docker Hub to pull the image
      - name: Log in to Docker Hub
        uses: docker/login-action@v2
        with:
          username: ${ secrets.DOCKER_USERNAME } # GitHub Secrets for Docker credentials
          password: ${ secrets.DOCKER_PASSWORD }

      # Step 3: Pull the Docker image from Docker Hub
      - name: Pull Docker image from Docker Hub
        run: |
          docker pull username/your-image:latest # Replace with your Docker image and tag

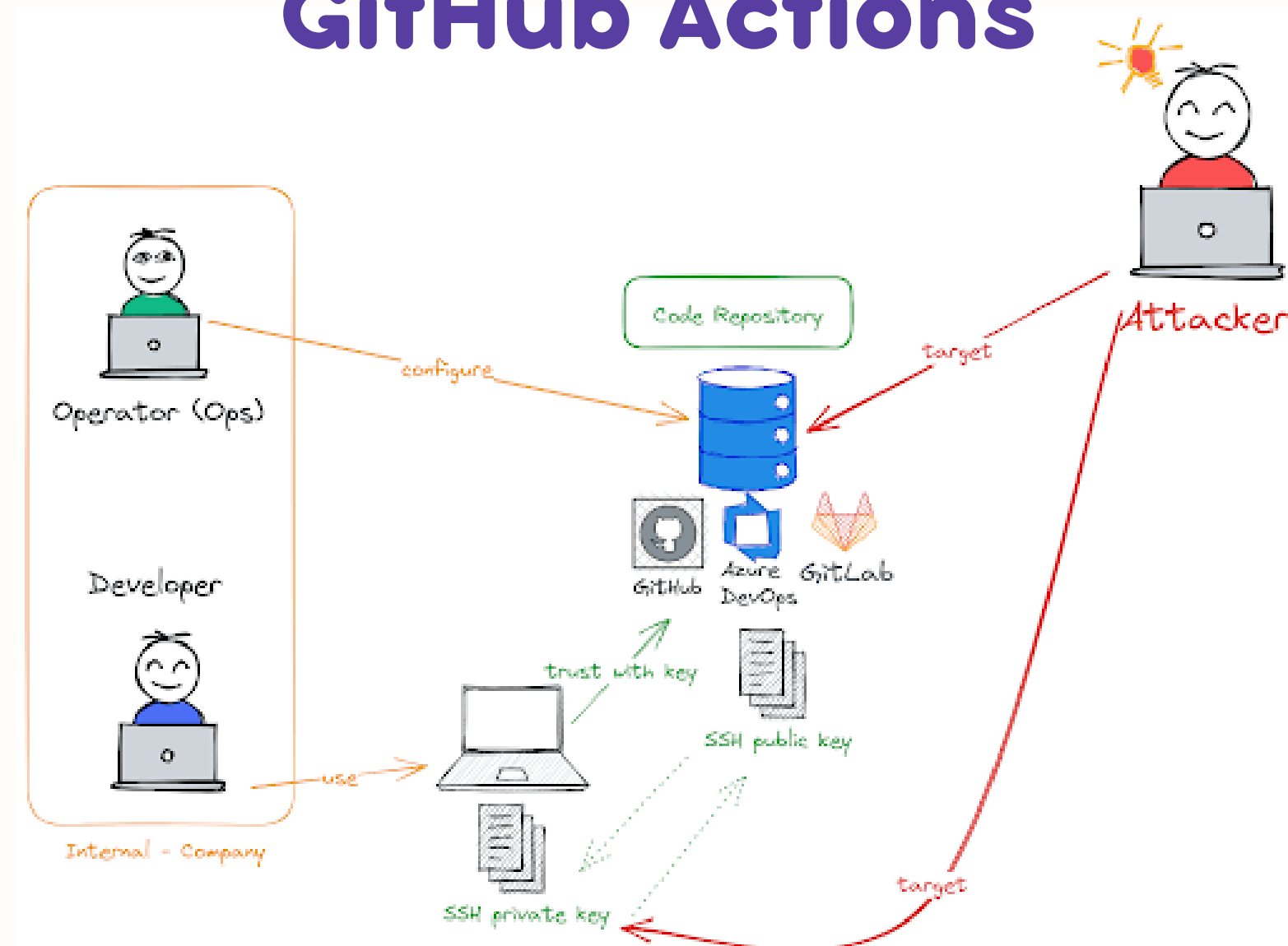
      # Step 4: Deploy the Docker container to Vercel (or any other service that supports Docker)
      # In this example, we will deploy the app via the Vercel CLI (you need Vercel CLI installed)
      - name: Deploy to Vercel
        env:
          VERCEL_TOKEN: ${ secrets.VERCEL_TOKEN } # Vercel API token stored in GitHub Secrets
        run: |
          # Install the Vercel CLI
          npm install -g vercel

          # Set up the project (you may need to adjust based on your Vercel setup)
          vercel login --token $VERCEL_TOKEN # Login to Vercel using the token
          vercel --prod # Deploy to production environment

      # Step 5: Optional - Notify Slack (or any other service) about the deployment status
      - name: Notify Slack about Deployment Status
        if: success() # This sends a Slack message only if the previous steps are successful
        uses: rtCamp/action-slack-notify@v2
        with:
          webhook_url: ${ secrets.SLACK_WEBHOOK_URL } # Slack webhook URL stored in GitHub Secrets
          message: 'Deployment to Vercel was successful!'

      # Optional: You can add more steps like notification or rollbacks if necessary
```

Mesures de sécurité essentielles pour GitHub Actions



Utiliser des Secrets GitHub pour stocker les clés SSH et autres informations sensibles

Limiter les permissions des clés SSH

Utiliser une connexion SSH sécurisée avec `known_hosts`

Activer la double authentification (2FA)

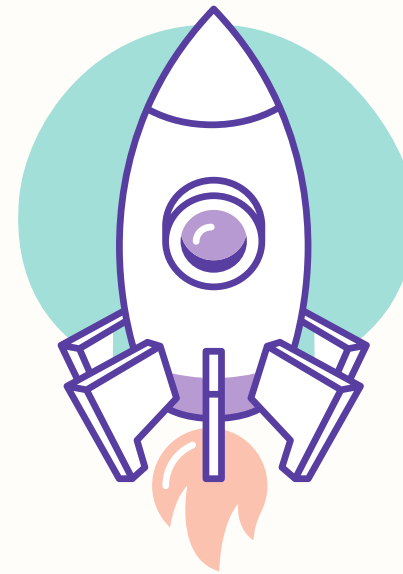
Pourquoi utiliser CI/CD au lieu de faire manuellement

Automatisation des tâches répétitives

Réduction des erreurs humaines dans la configuration

Vitesse et efficacité accrues

réutilisabilité

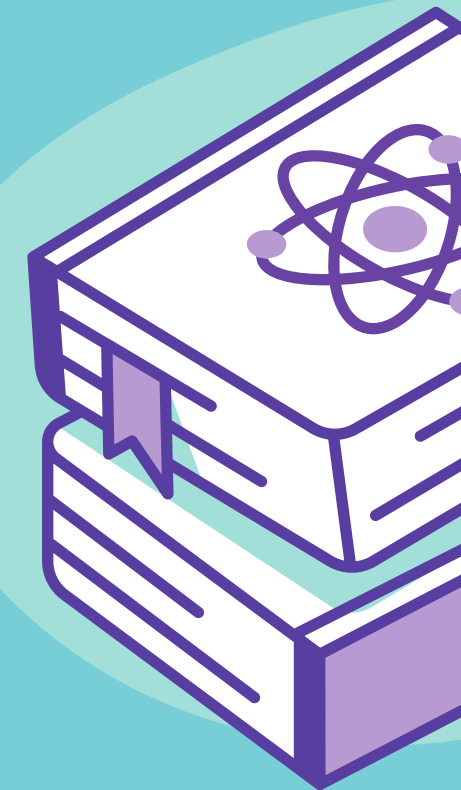




PARTIE 4

WORKSHOP PRATIQUE





THANK YOU FOR
YOUR
ATTENTION

